

LAPORAN TUGAS
ALJABAR LINEAR (PRAKTIKUM)
MODUL 1: PENGENALAN RUST DAN DASAR KOMPUTASI VEKTOR

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah
Mata Kuliah : Aljabar Linear (Praktikum)



Disusun Oleh :
Nama : Ghaisan Khoirul Badruzaman
NIM : 251524048
Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

DAFTAR ISI

BAB I PENDAHULUAN	4
BAB II SETUP ENVIRONMENT DAN VERIFIKASI TOOLCHAIN	6
2.1 Deskripsi Program	6
2.2 Konfigurasi Toolchain	6
2.3 Kode Program	7
2.4 Output Program	9
2.5 Lesson Learnt	9
BAB III DASAR SINTAKS DAN SISTEM TIPE RUST	11
3.1 Deskripsi Program	11
3.2 Definisi Data yang Digunakan	11
3.3 Kode Program	12
3.4 Penjelasan Proses	15
Bagian 1: Variable Binding dan Immutability	15
Bagian 2: Operasi Aritmetika i32 dan f64	15
Bagian 3: Array dan Indexing	16
Bagian 4: Perulangan dan Percabangan	16
3.5 Output Program	16
3.6 Lesson Learnt	18
BAB IV VEKTOR DENGAN ARRAY DAN VEC	19
4.1 Deskripsi Program	19
4.2 Definisi Data yang Digunakan	19
4.3 Keterbatasan Array Native	20
4.4 Kode Program	21
4.5 Penjelasan Proses	23
Fungsi tambah dan skalar	23

Fungsi dot	24
Fungsi statistik jumlah, rata, maks, dan mini	24
4.6 Output Program	24
4.7 Lesson Learnt	26
BAB V INTRODUKSI NALGEBRA UNTUK VEKTOR	27
5.1 Deskripsi Program	27
5.2 Definisi Data yang Digunakan	27
5.3 Kode Program	28
5.4 Penjelasan Proses	30
Pembuatan Vektor	30
Aritmetika Element-wise lewat Operator	30
Dot Product, Norma, dan Sudut	31
Fungsi Statistik	31
Latihan Terapan	32
5.5 Output Program	32
5.6 Perbandingan dengan Implementasi Manual	35
5.7 Lesson Learnt	35
BAB VI PENGUJIAN OTOMATIS DENGAN CARGO TEST	37
6.1 Deskripsi Pengujian	37
6.2 Daftar Unit Test	37
6.3 Perbandingan Floating Point	39
6.4 Hasil Pengujian	39
6.5 Kualitas Kode	41
BAB VII KESIMPULAN	42

BAB I PENDAHULUAN

Aljabar linear menjadi fondasi banyak bidang komputasi, mulai dari pembelajaran mesin, grafika komputer, sampai sistem tertanam, tetapi konsepnya mudah berhenti sebagai rumus di papan tulis kalau tidak pernah dijalankan. Menguji konsep vektor secara komputasional membuat sisi aljabaris dan intuisi geometrisnya terbentuk bersamaan, karena hasil hitungan program bisa langsung dibandingkan dengan hitungan tangan. Modul 1 memakai Rust, bukan Python dengan NumPy, karena Rust menggabungkan performa setara C dengan jaminan memory safety yang diperiksa saat kompilasi. Kombinasi tersebut cocok untuk kode numerik yang harus andal dan dapat diuji, sebab kesalahan tipe maupun kesalahan akses memori tertangkap sebelum program sempat berjalan. Pemilihan Rust juga membiasakan disiplin tipe, ownership, dan pengujian otomatis sejak modul pertama, bukan setelah kodenya terlanjur besar.

Sasaran praktikum Modul 1 terbagi menjadi empat hal mengikuti ruang lingkup pada bagian I.2 modul. Pertama, mengonfigurasi toolchain Rust berupa rustup, cargo, dan rust-analyzer sampai sebuah proyek cargo dapat dikompilasi dan dijalankan tanpa warning. Kedua, menguasai dasar sintaks dan sistem tipe Rust yang mencakup i32, f64, array, slice, $\text{Vec}<T>$, struktur kontrol loop, while, for, dan if, definisi fungsi, serta ownership dasar. Ketiga, merepresentasikan vektor memakai array, slice, dan Vec native beserta operasi manual berbasis iterator, sekaligus menunjukkan keterbatasannya terhadap operator matematika. Keempat, mengenal crate nalgebra sebagai pembandingnya lewat operator overloading, dot product, norma, dan fungsi statistik, dengan seluruh hasilnya divalidasi memakai cargo test.

Vektor di $\mathbb{R}^n$ didefinisikan sebagai sekumpulan komponen terurut $v = (v_1, v_2, \dots, v_n)$, yang secara komputasional dimodelkan sebagai blok memori homogen. Penjumlahan dua vektor dikerjakan per komponen, $u + v = (u_1+v_1, u_2+v_2, \dots, u_n+v_n)$, sedangkan perkalian dengan skalar k mengalikan tiap komponen sehingga $k*v = (k*v_1, k*v_2, \dots, k*v_n)$. Dot product menghasilkan sebuah skalar dari jumlah hasil kali komponen yang bersesuaian, $u \cdot v = u_1*v_1 + u_2*v_2 + \dots + u_n*v_n$. Nilai itu terhubung dengan geometri lewat relasi $u \cdot v = |u| |v| \cos \theta$, sehingga sudut antara dua vektor bisa dicari dengan $\theta = \arccos(u \cdot v / (|u| |v|))$. Norma atau panjang vektor didefinisikan sebagai $|v| = \sqrt{v \cdot v}$, dan norma inilah yang menjadi penyebut pada relasi sudut tersebut.

Rust bersifat statically typed, seluruh tipe diperiksa saat kompilasi sehingga ketidakcocokan tipe tidak pernah lolos sampai runtime. Setiap nilai memiliki tepat satu owner, dan akses dari tempat lain harus melalui borrow, baik $\&T$ yang immutable maupun $\&\text{mut } T$ yang mutable. Aturan tersebut menutup data race dan use-after-free tanpa garbage collector, jadi ongkos keamanannya dibayar pada waktu kompilasi, bukan pada waktu eksekusi. nalgebra memanfaatkan sistem tipe

yang sama dengan mengodekan dimensi vektor sebagai bagian dari tipenya, misalnya `Vector3<f64>` yang panjangnya pasti tiga dan `SVector<f64, 4>` yang panjangnya pasti empat. Konsekuensinya, operasi antara dua vektor berdimensi berbeda ditolak kompilator, berbeda dengan pendekatan dinamis NumPy yang baru melempar galat ketika baris bersangkutan dieksekusi.

Laporan ini disusun dalam tujuh bab. BAB II menguraikan penyiapan lingkungan kerja beserta Task 0 berupa verifikasi toolchain dan crate nalgebra. BAB III membahas Task I mengenai dasar sintaks dan sistem tipe, BAB IV membahas Task II mengenai vektor dengan array dan `Vec` secara manual, lalu BAB V membahas Task III mengenai nalgebra. BAB VI merangkum pengujian otomatis lewat cargo test yang menjalankan 17 unit test, dan BAB VII berisi kesimpulan serta lesson learnt dari keseluruhan praktikum.

BAB II SETUP ENVIRONMENT DAN VERIFIKASI TOOLCHAIN

2.1 Deskripsi Program

Task 0 meminta pembuatan proyek cargo baru bernama praktikum-al, pengisian src/main.rs dengan dua baris cetak, penambahan nalgebra 0.35.0 sebagai dependensi, lalu pembuktian bahwa crate tersebut benar terpasang lewat pembuatan satu Vector3 yang langsung dicetak. Identitas penulis ditulis sebagai komentar pada baris paling atas berkas, sesuai poin kelima instruksi modul. Program yang dihasilkan tidak menghitung apa pun; keluarannya hanya sapaan, penanda bahwa rustc berhasil mengompilasi, dan wujud sebuah vektor yang dirender oleh nalgebra. Yang diuji di sini adalah rantai perkakasnya, bukan logika numeriknya.

Verifikasi semacam ini dikerjakan lebih dulu karena kegagalan build punya dua sumber yang gejalanya mirip, yaitu kesalahan pada kode dan kerusakan pada toolchain atau crate yang gagal terunduh. Bila Task I sampai Task III langsung ditulis tanpa pengecekan awal, satu pesan error dari cargo bisa berarti salah sintaks, bisa juga berarti registry crates.io tidak terjangkau. Task 0 memisahkan dua kemungkinan itu dengan program sekecil mungkin sehingga bila ia jalan, semua kegagalan setelahnya pasti berasal dari kode yang ditulis sendiri. Alasan kedua menyangkut versi, sebab API nalgebra berubah antar rilis minor dan edition 2024 memberlakukan aturan yang berbeda dari edition sebelumnya, sehingga versi yang dipakai perlu dicatat sejak awal agar hasil pada bab berikutnya dapat ditelusuri ulang.

2.2 Konfigurasi Toolchain

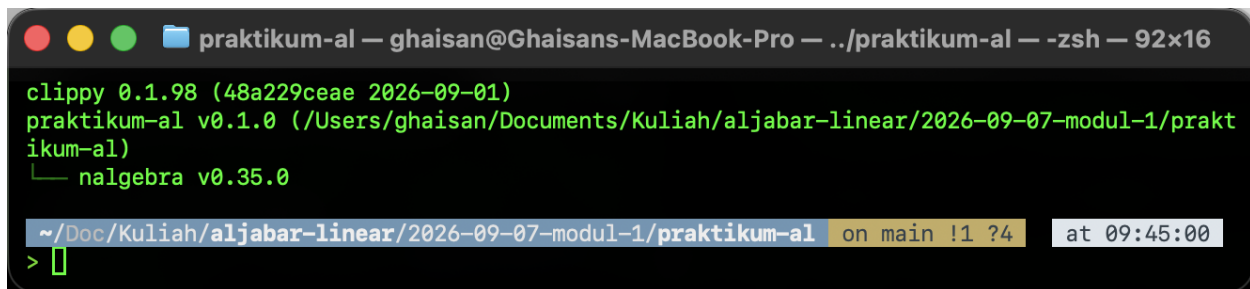
Pemasangan dilakukan lewat rustup, skrip resmi yang diunduh dari sh.rustup.rs dan dijalankan langsung di terminal. Yang terpasang bukan rustc secara telanjang melainkan rustup sebagai manajer toolchain, yang kemudian menarik channel stable beserta rustc, cargo, rustfmt, dan clippy sekaligus. Seluruh binary diletakkan di ~/.cargo/bin sehingga direktori itu perlu masuk PATH, biasanya dengan menjalankan source "\$HOME/.cargo/env" atau membuka ulang terminal. Pada MacBook Pro M2 Pro yang dipakai, rustup memilih target aarch64-apple-darwin secara otomatis dari deteksi arsitektur host. Keuntungan memakai rustup adalah beberapa toolchain bisa berdampingan dan perpindahan versi tidak merusak instalasi yang lain.

Cargo memegang dua peran sekaligus. Sebagai build system ia yang memanggil rustc dengan flag yang benar, mengatur urutan kompilasi, dan menyimpan hasil antara di direktori target sehingga build berikutnya hanya mengulang bagian yang berubah; perintah cargo build, cargo run, cargo test, cargo fmt, dan cargo clippy semuanya masuk lewat pintu ini. Sebagai package manager ia membaca daftar dependensi yang ditulis deklaratif di Cargo.toml, mengunduhnya dari crates.io, lalu menyelesaikan dependensi transitif yang ikut terbawa. Menambahkan nalgebra

cukup dengan satu baris `nalgebra = "0.35.0"` di bawah bagian `[dependencies]`, setelah itu cargo build yang mengurus sisanya termasuk menarik `approx`, `num-traits`, `simba`, dan crate turunan lain yang dibutuhkan `nalgebra`. Nama proyek, versi, dan edition 2024 ditulis di bagian `[package]` pada berkas yang sama.

```
[package]
name = "praktikum-al"
version = "0.1.0"
edition = "2024"

[dependencies]
nalgebra = "0.35.0"
```



Gambar 2.1: Verifikasi versi `rustc`, `cargo`, `clippy`, dan `nalgebra`

Versi yang terpasang pada mesin praktikum adalah `rustc` 1.98.1, `cargo` 1.98.1, dan `clippy` 0.1.98, ketiganya berasal dari channel stable yang sama. Versi `nalgebra` dibaca dari `Cargo.lock`, bukan dari `Cargo.toml`, karena berkas manifest hanya menyatakan permintaan sedangkan berkas kunci mencatat apa yang benar-benar dipakai, dan hasilnya adalah `nalgebra` 0.35.0. Modul mensyaratkan `rustc` minimal 1.75 pada bagian Development Tools, sehingga syarat itu terpenuhi dengan selisih yang cukup jauh. Jarak versi tersebut juga menjadi alasan edition 2024 bisa dipakai tanpa masalah, sebab edition itu baru didukung penuh oleh `rustc` rilis yang jauh di atas 1.75.

2.3 Kode Program

Isi lengkap `src/main.rs` setelah seluruh task selesai dikerjakan dikutip di bawah ini.

```
// Nama : Ghaisan Khoirul Badruzaman
// NIM : 251524048
// Kelas : 2B-D4
// Modul 1: Pengenalan Rust & Dasar Komputasi Vektor

mod task1;
mod task2;
mod task3;

use nalgebra::Vector3;

/// Task 0: memastikan rustc, cargo, dan crate nalgebra benar-benar terpasang.
fn task0() {
    println!("Hello, Aljabar Linear!");
```

```

println!("Rust version check: OK");

let v = Vector3::new(1.0, 2.0, 3.0);
println!("nalgebra OK, contoh vektor: {v}");
}

fn main() {
    // Argumen opsional dipakai buat menjalankan satu task saja waktu mendemokan
    hasilnya.
    // Tanpa argumen, keempat task jalan berurutan.
    match std::env::args().nth(1).as_deref() {
        Some("0") => task0(),
        Some("1") => task1::jalankan(),
        Some("2") => task2::jalankan(),
        Some("3") => task3::jalankan(),
        _ => {
            task0();
            task1::jalankan();
            task2::jalankan();
            task3::jalankan();
        }
    }
}

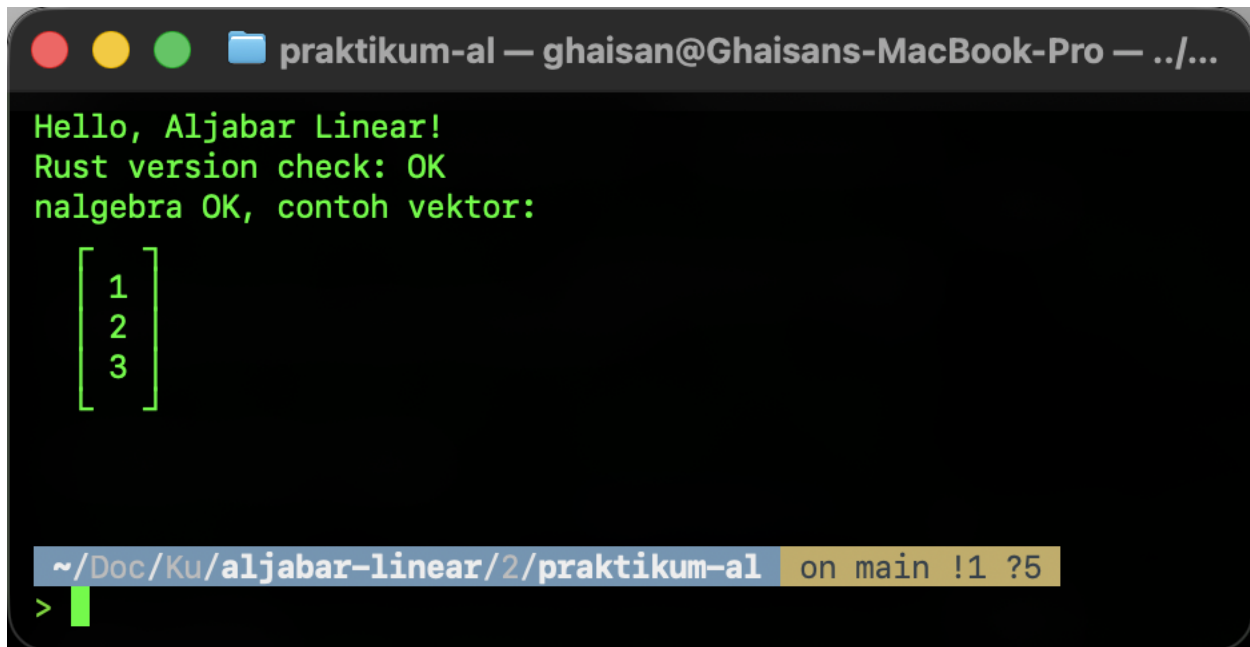
```

Tiga baris `mod task1`, `mod task2`, dan `mod task3` mendaftarkan berkas `src/task1.rs`, `src/task2.rs`, dan `src/task3.rs` sebagai modul milik crate ini. Tanpa deklarasi tersebut ketiga berkas itu tidak akan ikut dikompilasi sama sekali meski sudah ada di dalam direktori `src`, karena Rust menyusun pohon modul dari deklarasi eksplisit, bukan dari isi direktori. Baris `use nalgebra::Vector3` menarik satu tipe dari crate eksternal ke lingkup berkas ini supaya tidak perlu ditulis dengan jalur panjang setiap kali dipakai. Fungsi `task0()` sendiri hanya berisi dua panggilan `println!` lalu pembuatan vektor lewat `Vector3::new(1.0, 2.0, 3.0)`, yang tipe elemennya disimpulkan sebagai `f64` dari bentuk literal desimalnya. Penulisan `{v}` di dalam string format adalah inline format argument, cara ringkas memanggil implementasi `Display` milik `Nalgebra` tanpa menyebut variabelnya lagi sebagai argumen terpisah.

Fungsi `main()` tidak langsung memanggil keempat task, melainkan mencocokkan argumen baris perintah lebih dulu. Ekspresi `std::env::args()` mengembalikan iterator berisi seluruh argumen dengan nama program pada indeks nol, sehingga `nth(1)` mengambil argumen pertama yang diketik pengguna dan menghasilkan `Option<String>`. Pemanggilan `as_deref()` mengubah nilai itu menjadi `Option<&str>` supaya bisa dicocokkan langsung dengan pola `Some("0")` sampai `Some("3")`, sebab literal string dalam pola bertipe `&str` dan bukan `String`. Arm terakhir dengan pola `_` menangkap semua kemungkinan lain, termasuk kondisi tanpa argumen sama sekali, lalu menjalankan keempat task berurutan. Percabangan ini dipakai supaya saat mengambil tangkapan layar untuk tiap bab, keluaran satu task tidak tercampur dengan task lain, dan pemanggilannya cukup dengan `cargo run -- 0`.

2.4 Output Program

Hasil eksekusi `cargo run -- 0` pada terminal ditampilkan pada gambar berikut.

A screenshot of a macOS terminal window titled "praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../...". The terminal displays the following output in green text: "Hello, Aljabar Linear!", "Rust version check: OK", and "nalgebra OK, contoh vektor:". Below the text is a column vector matrix with elements 1, 2, and 3. The terminal prompt is "~/Doc/Ku/aljabar-linear/2/praktikum-al" on the main branch, with a cursor at the end.

```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../...  
Hello, Aljabar Linear!  
Rust version check: OK  
nalgebra OK, contoh vektor:  

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$
  
~/Doc/Ku/aljabar-linear/2/praktikum-al on main !1 ?5  
>
```

Gambar 2.2: Hasil cargo run untuk Task 0

Dua baris teks pertama, Hello, Aljabar Linear! dan Rust version check: OK, membuktikan rustc berhasil mengompilasi berkas dan binary hasilnya benar dieksekusi. Baris ketiga diawali teks nalgebra OK, contoh vektor: lalu diikuti vektor yang dicetak nalgebra dalam bentuk matriks kolom berbingkai, dengan nilai 1, 2, dan 3 tersusun dari atas ke bawah. Angka tampil tanpa bagian desimal walaupun tipenya `f64`, karena implementasi `Display` milik nalgebra memangkas nol di belakang koma agar keluaran tetap ringkas. Bentuk kolom itu sendiri sudah menyiratkan bahwa `Vector3` di nalgebra sebenarnya adalah matriks berukuran tiga baris satu kolom, hal yang akan dipakai lagi pada bab tentang operasi vektor.

2.5 Lesson Learnt

Memastikan toolchain benar sebelum mulai menulis kode terbukti bukan formalitas. Saat menyiapkan environment, rustup sempat dijalankan dua kali secara bersamaan di dua jendela terminal, satu untuk update dan satu untuk memasang komponen tambahan. Kedua proses menulis ke direktori `~/.rustup` yang sama, akibatnya metadata toolchain rusak dan setiap pemanggilan cargo berikutnya berhenti dengan galat `missing manifest`. Perbaikannya adalah menghapus toolchain stable lalu memasangnya ulang dari awal, dan setelah itu semua perintah kembali normal. Pelajaran praktisnya, rustup tidak aman dijalankan paralel, jadi satu operasi harus selesai sebelum operasi berikutnya dimulai.

Hal kedua menyangkut perbedaan peran antara Cargo.toml dan Cargo.lock. Baris `nalgebra = "0.35.0"` pada manifest sebenarnya menyatakan rentang versi yang bisa diterima, bukan satu titik pasti, sehingga build di waktu berbeda berpotensi mengambil rilis patch yang lebih baru. Cargo.lock menutup celah itu dengan mencatat versi persis setiap crate sampai ke dependensi transitif seperti `approx 0.5.1` dan `num-traits`, lengkap dengan checksum masing-masing. Selama berkas kunci itu ikut disimpan bersama proyek, kompilasi ulang di mesin lain akan menghasilkan pohon dependensi yang identik, jadi angka yang dilaporkan pada bab berikutnya dapat direproduksi apa adanya.

BAB III DASAR SINTAKS DAN SISTEM TIPE RUST

3.1 Deskripsi Program

Task I dikerjakan sebagai modul terpisah pada berkas `src/task1.rs`, dideklarasikan lewat `mod task1`; di `main.rs` dan dipanggil melalui `task1::jalankan()` ketika program dijalankan dengan argumen 1. Isinya empat bagian eksplorasi yang dicetak berurutan ke terminal, masing-masing dipisahkan garis pembatas oleh fungsi bantu `judul()`. Selain fungsi utama `jalankan()`, berkas ini memuat dua fungsi pendukung, yaitu `hampir_sama()` untuk membandingkan dua nilai `f64` dan `klasifikasi()` untuk menentukan sebuah nilai genap atau ganjil. Pemisahan itu membuat logika yang dibahas bisa dipanggil ulang dari modul `tests` di bagian bawah berkas.

Yang dieksplorasi adalah fondasi bahasa sebelum masuk ke komputasi vektor pada task berikutnya. Bagian pertama menunjukkan cara Rust mengikat nilai ke nama beserta sifat `immutable` bawaannya, bagian kedua membandingkan perilaku operator aritmetika pada `i32` dan `f64`, bagian ketiga menyentuh array panjang tetap beserta `indexing` dan `slicing`, dan bagian keempat menggabungkan perulangan dengan percabangan. Konstanta `EPSILON` ditambahkan di luar daftar minimal modul karena perbandingan bilangan pecahan memakai operator sama dengan mudah meleset, dan konstanta itu langsung dipakai untuk memeriksa apakah $0.1 + 0.2$ sama dengan 0.3 . Lima unit test di akhir berkas mengunci perilaku yang dijelaskan pada bab ini supaya tidak berubah diam-diam ketika kode disunting.

3.2 Definisi Data yang Digunakan

Binding `n` bertipe `i32` diberi nilai 10. Tipe `i32` adalah bilangan bulat bertanda 32-bit dan menurut Tabel 3 modul mendukung operator `+` `-` `*` `/` `%` beserta operasi bitwise. Sifat yang paling terasa pada task ini adalah pembagiannya, sebab `/` pada dua nilai `i32` menghasilkan `i32` lagi sehingga bagian pecahan hasilnya dibuang. Literal 7 dan 2 pada baris `7 / 2` juga bertipe `i32` karena Rust memilih `i32` sebagai tipe integer bawaan ketika tidak ada anotasi maupun sufiks.

Nilai desimal diwakili `phi` bertipe `f64` dengan nilai 3.14159, yaitu bilangan titik-mengambang presisi ganda 64-bit. Selain lima operator aritmetika yang sama dengan integer, Tabel 3 mencatat `f64` punya `powf`, `sqrt`, dan `abs` sebagai method. Ketiganya terpakai di berkas ini, yaitu `powf` pada `a.powf(b)`, `sqrt` pada `9.0f64.sqrt()`, dan `abs` di dalam `hampir_sama()` untuk mengambil nilai mutlak selisih dua bilangan. Pasangan `a` dan `b` sengaja ditulis `7.0f64` dan `2.0f64` memakai sufiks tipe supaya kontrasnya dengan versi integer `7 / 2` terlihat jelas.

Tipe `bool` diwakili aktif yang bernilai `true` dan hanya mengenal operator logika `&&`, `||`, serta `!` sesuai Tabel 3. Perannya bukan sebatas data yang dicetak, sebab `hampir_sama()` mengembalikan `bool` dan nilai itulah yang menentukan cabang mana yang diambil pernyataan `if` di dalam

klasifikasi(). Untuk teks, binding nama bertipe &str berisi "Vektor", sebuah string slice yang menunjuk data teks tanpa memilikinya, dengan operasi terbatas pada slicing dan penggabungan lewat format!. Tipe yang sama muncul pada parameter fungsi judul(teks: &str) serta pada nilai balik klasifikasi() yang bertipe &'static str, karena literal "genap" dan "ganjil" tertanam di binary sepanjang program hidup.

Konstanta EPSILON dideklarasikan dengan const EPSILON: f64 = 1e-10; di lingkup modul, di luar fungsi mana pun. Berbeda dari let, const wajib punya anotasi tipe, nilainya harus bisa dihitung saat kompilasi, dan tidak bisa ditandai mut sama sekali. Angka 1e-10 dipilih sebagai ambang toleransi yang masih jauh lebih besar dari galat pembulatan f64 pada operasi sederhana, tetapi cukup kecil untuk menolak dua bilangan yang memang berbeda. Karena bertipe f64, konstanta ini bisa langsung ikut operasi pengurangan dan perbandingan di dalam hampir_sama().

Data numerik utama disimpan pada v bertipe [f64; 5] berisi 10.0 sampai 50.0. Panjang array ikut menjadi bagian dari tipenya, jadi ukurannya sudah pasti saat kompilasi dan datanya duduk di stack. Operasi yang tersedia menurut Tabel 3 adalah indexing, iterasi, dan map lewat iter(), tanpa satu pun operator aritmetika antar array. Dari array yang sama diambil potongan &v[1..4] bertipe &[f64], sedangkan array kecil [3.0, 4.0, 7.0] pada bagian keempat bertipe [f64; 3] yang secara compiler dianggap tipe berbeda meski elemennya sama-sama f64.

3.3 Kode Program

Isi lengkap berkas src/task1.rs adalah sebagai berikut.

```
// Nama : Ghaisan Khoirul Badruzaman
// NIM : 251524048
// Kelas : 2B-D4
// Modul 1 - Task I: Dasar Sintaks & Tipe Data Rust

/// Ambang toleransi buat membandingkan dua f64.
const EPSILON: f64 = 1e-10;

fn judul(teks: &str) {
    println!("\n{}", "-".repeat(56));
    println!("{}", teks);
    println!("{}", "-".repeat(56));
}

/// Perbandingan float pakai == gampang meleset karena galat pembulatan,
/// jadi selisihnya yang dicek terhadap EPSILON.
fn hampir_sama(a: f64, b: f64) -> bool {
    (a - b).abs() < EPSILON
}

fn klasifikasi(nilai: f64) -> &'static str {
    if nilai as i64 % 2 == 0 {
        "genap"
    } else {

```

```

        "ganjil"
    }
}

pub fn jalankan() {
    judul("Bagian 1: Variable Binding & Immutability");
    let n: i32 = 10;
    #[allow(clippy::approx_constant)] // 3.14159 mendekati PI, tanpa ini clippy
menolak
    let phi: f64 = 3.14159;
    let aktif: bool = true;
    let nama: &str = "Vektor";
    println!("n (i32) = {n}");
    println!("phi (f64) = {phi}");
    println!("aktif (bool) = {aktif}");
    println!("nama (&str) = {nama}");

    // binding default immutable, mau diubah nilainya harus ditandai mut dulu
    let tetap = 5;
    let mut berubah = 5;
    berubah += 3;
    println!("tetap (tanpa mut, nilainya kunci) = {tetap}");
    println!("berubah (pakai mut, setelah += 3) = {berubah}");
    println!("EPSILON (const) = {EPSILON:e}");
    println!(
        "0.1 + 0.2 hampir sama 0.3 ? = {}",
        hampir_sama(0.1 + 0.2, 0.3)
    );

    judul("Bagian 2: Operasi Aritmetika i32 dan f64");
    let (a, b) = (7.0f64, 2.0f64);
    println!("a / b = {}", a / b);
    println!("a % b = {}", a % b);
    println!("a.powf(b) = {}", a.powf(b));
    // pembagian antar i32 membuang bagian pecahannya, bukan dibulatkan
    println!("7 / 2 (i32) = {}", 7 / 2);
    println!("7 % 2 (i32) = {}", 7 % 2);
    // sqrt cuma ada di tipe float, integer harus di-cast dulu
    println!("9.0.sqrt() = {}", 9.0f64.sqrt());

    judul("Bagian 3: Array & Indexing");
    let v: [f64; 5] = [10.0, 20.0, 30.0, 40.0, 50.0];
    let potongan = &v[1..4];
    println!("v = {v:?}");
    println!("v[0] = {}", v[0]);
    println!("v[len - 1] = {}", v[v.len() - 1]);
    println!("&v[1..4] = {potongan:?}");

    judul("Bagian 4: Perulangan & Percabangan");
    for nilai in v {
        println!("{nilai} -> {}", klasifikasi(nilai));
    }

    // Isi v kebetulan genap semua, jadi cabang else belum terbukti jalan.
    // Array kecil ini dipakai supaya kedua cabang sama-sama tercetak.
    for nilai in [3.0, 4.0, 7.0] {
        println!("{nilai} -> {}", klasifikasi(nilai));
    }
}

```

```

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn klasifikasi_genap_ganjil() {
        assert_eq!(klasifikasi(10.0), "genap");
        assert_eq!(klasifikasi(40.0), "genap");
        assert_eq!(klasifikasi(25.0), "ganjil");
        assert_eq!(klasifikasi(-3.0), "ganjil");
    }

    #[test]
    fn hampir_sama_menoleransi_galat_float() {
        assert!(hampir_sama(0.1 + 0.2, 0.3));
        assert!(!hampir_sama(1.0, 1.001));
    }

    #[test]
    fn pembagian_i32_terpotong() {
        assert_eq!(7 / 2, 3);
        assert_eq!(7 % 2, 1);
        assert!(hampir_sama(7.0 / 2.0, 3.5));
    }

    #[test]
    fn operasi_float_dasar() {
        assert!(hampir_sama(7.0f64.powf(2.0), 49.0));
        assert!(hampir_sama(9.0f64.sqrt(), 3.0));
        assert!(hampir_sama(7.0f64 % 2.0, 1.0));
    }

    #[test]
    fn indexing_dan_slicing() {
        let v: [f64; 5] = [10.0, 20.0, 30.0, 40.0, 50.0];
        assert_eq!(v.len(), 5);
        assert!(hampir_sama(v[0], 10.0));
        assert!(hampir_sama(v[v.len() - 1], 50.0));
        assert_eq!(&v[1..4], &[20.0, 30.0, 40.0]);
    }
}

```

3.4 Penjelasan Proses

Bagian 1: Variable Binding dan Immutability

Empat binding pertama ditulis dengan anotasi tipe eksplisit meski Rust sebenarnya sanggup menyimpulkannya sendiri, tujuannya supaya tipe yang sedang dibahas terbaca langsung di kode. Atribut `#[allow(clippy::approx_constant)]` di atas `phi` diperlukan karena `clippy` punya lint `approx_constant` yang menandai literal pecahan bernilai dekat dengan konstanta matematika bawaan lalu menyarankan pemakaian `std::f64::consts::PI`. Angka 3.14159 masuk rentang deteksi lint tersebut, padahal modul memang meminta literal itu ditulis apa adanya, sehingga atributnya dipasang tepat di atas satu pernyataan yang bermasalah saja dan tidak mematikan lint untuk seluruh berkas. Tanpa atribut ini perintah `cargo clippy --all-targets` akan mengeluarkan peringatan dan syarat `clippy` bersih tidak terpenuhi.

Pasangan tetap dan berubah dipakai untuk menunjukkan immutability default. Keduanya diberi nilai awal 5, tetapi hanya berubah yang ditandai `mut` sehingga boleh menerima `+= 3` dan tercetak menjadi 8, sedangkan tetap akan ditolak compiler begitu ada percobaan menugaskan ulang nilainya. Baris berikutnya mencetak EPSILON memakai format `{:e}` sehingga tampil sebagai `1e-10`, lalu memanggil `hampir_sama(0.1 + 0.2, 0.3)` yang menghasilkan `true`. Nilai `true` itu tidak berarti kedua sisi identik bit per bit, melainkan selisihnya lebih kecil dari ambang EPSILON.

Bagian 2: Operasi Aritmetika i32 dan f64

Pasangan `a` dan `b` bertipe `f64` dijalankan lewat tiga operasi, yaitu `a / b` yang memberi 3.5, `a % b` yang memberi sisa 1, dan `a.powf(b)` yang memberi 49. Dua hasil terakhir tercetak tanpa koma desimal karena implementasi `Display` untuk `f64` menghilangkan bagian pecahan yang bernilai nol, jadi 1.0 tampil sebagai 1 dan 49.0 tampil sebagai 49. Berbeda dengan itu, `7 / 2` menghasilkan 3 karena kedua operannya literal `i32` sehingga compiler memilih pembagian integer, dan pembagian integer memotong bagian pecahan ke arah nol alih-alih membulatkan. Nilai 3.5 kehilangan 0.5-nya di titik itu, dan sisa yang hilang tersebut justru yang muncul pada `7 % 2` yang bernilai 1.

Baris `9.0f64.sqrt()` memakai sufiks `f64` pada literalnya karena method `sqrt` hanya didefinisikan untuk tipe titik-mengambang, yakni `f32` dan `f64`, dan tidak tersedia pada `i32`. Alasannya sederhana, akar kuadrat bilangan bulat pada umumnya bukan bilangan bulat, jadi tidak ada nilai balik yang masuk akal kalau method itu dipasang pada tipe integer. Bila yang mau diakarkan berupa integer, nilainya harus di-cast dulu dengan `as f64` sebelum `sqrt` bisa dipanggil. Hasilnya di terminal tercetak 3 dengan alasan format yang sama seperti pada `powf` tadi.

Bagian 3: Array dan Indexing

Array `v` dideklarasikan sebagai `[f64; 5]` dan diakses dengan indexing bergaya `v[0]` yang bernilai 10 serta `v[v.len() - 1]` yang bernilai 50. Rust tidak mengenal indeks negatif, sehingga elemen terakhir harus dihitung manual dari panjangnya, dan indeks yang dipakai bertipe `usize`. Akses di luar rentang tidak lolos begitu saja seperti pada C, melainkan memicu panic dengan pesan yang jelas saat program berjalan.

Potongan `&v[1..4]` adalah slice bertipe `&[f64]`, yaitu pinjaman read-only atas sebagian isi `v`. Yang disimpan slice hanya pointer ke elemen awal beserta panjangnya, jadi tidak ada penyalinan data dan `v` tetap menjadi pemilik isinya. Karena bentuknya `&` tanpa `mut`, isi yang ditunjuk tidak bisa diubah lewat potongan tersebut, dan selama potongan itu masih terpakai compiler juga melarang `v` dipinjam secara mutable. Rentang `1..4` bersifat setengah terbuka sehingga yang terambil indeks 1, 2, dan 3, dan hasilnya `[20.0, 30.0, 40.0]` dicetak dengan `{:?}` sebab array dan slice mengimplementasikan `Debug`, bukan `Display`.

Bagian 4: Perulangan dan Percabangan

Perulangan `for` nilai `in v` mengambil elemen array by value, bukan by reference, karena `f64` bertipe `Copy` dan array sendiri mengimplementasikan `IntoIterator` yang menyerahkan elemennya. Setiap nilai diteruskan ke `klasifikasi()` yang melakukan cast nilai `as i64` lebih dulu, sebab operator `%` pada `f64` tetap menghasilkan `f64` dan pemeriksaan genap ganjil jadi kurang bersih dibanding pada bilangan bulat. Setelah cast, sisa bagi terhadap 2 dievaluasi di dalam `if` dan cabang yang terpilih mengembalikan salah satu dari dua literal string.

Isi `v` yang diminta modul kebetulan kelipatan sepuluh semua, jadi cabang `else` pada `klasifikasi()` tidak pernah tersentuh kalau hanya array itu yang diiterasi. Karena itu ditambahkan satu perulangan lagi atas array `[3.0, 4.0, 7.0]` yang memuat dua nilai ganjil dan satu nilai genap, sehingga kedua cabang percabangan sama-sama terbukti dieksekusi pada output. Perilaku yang sama dikunci pula di modul `tests` lewat `klasifikasi_genap_ganjil` yang menguji 10.0, 40.0, 25.0, dan -3.0.

3.5 Output Program

Hasil eksekusi perintah `cargo run -- 1` ditampilkan pada gambar berikut.


```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../pr...

Finished `dev` profile [unoptimized + debuginfo] target(s) in
0.01s
Running `target/debug/praktikum-al 1`

-----
Bagian 1: Variable Binding & Immutability
-----
n      (i32) = 10
phi    (f64) = 3.14159
aktif  (bool) = true
nama   (&str) = Vektor
tetap  (tanpa mut, nilainya kunci) = 5
berubah (pakai mut, setelah += 3) = 8
EPSILON (const) = 1e-10
0.1 + 0.2 hampir sama 0.3 ? = true

-----
Bagian 2: Operasi Aritmetika i32 dan f64
-----
a / b      = 3.5
a % b      = 1
a.powf(b)  = 49
7 / 2 (i32) = 3
```

Gambar 3.1a: Hasil cargo run untuk Task I, bagian atas

```
7 % 2 (i32) = 1
9.0.sqrt() = 3

-----
Bagian 3: Array & Indexing
-----
v      = [10.0, 20.0, 30.0, 40.0, 50.0]
v[0]   = 10
v[len - 1] = 50
&v[1..4] = [20.0, 30.0, 40.0]

-----
Bagian 4: Perulangan & Percabangan
-----
10 -> genap
20 -> genap
30 -> genap
40 -> genap
50 -> genap
3 -> ganjil
4 -> genap
7 -> ganjil

~/Doc/Ku/aljabar-linear/2/praktikum-al on main !1 ?5
> █
```

Gambar 3.1b: Hasil cargo run untuk Task I, bagian bawah

Keempat bagian tercetak berurutan mengikuti urutan pemanggilannya di dalam jalankan(). Bagian pertama menampilkan $n = 10$, $\phi = 3.14159$, $\text{aktif} = \text{true}$, dan $\text{nama} = \text{Vektor}$, disusun tetap yang bertahan di 5 sementara berubah menjadi 8, EPSILON tercetak $1e-10$, dan perbandingan $0.1 + 0.2$ terhadap 0.3 bernilai true. Bagian kedua memperlihatkan kontras yang dicari, yaitu $a / b = 3.5$ pada f64 melawan $7 / 2 = 3$ pada i32, dilengkapi $a \% b = 1$, $a.\text{powf}(b) = 49$, $7 \% 2 = 1$, dan $9.0.\text{sqrt}() = 3$. Bagian ketiga mencetak $v = [10.0, 20.0, 30.0, 40.0, 50.0]$, $v[0] = 10$, $v[\text{len} - 1] = 50$, dan $\&v[1..4] = [20.0, 30.0, 40.0]$. Pada bagian keempat, lima nilai pertama pada array v semuanya genap sehingga hanya cabang if yang terpakai, karena itu ditambahkan array kecil $[3.0, 4.0, 7.0]$ yang membuat 3 dan 7 terbaca ganjil sedangkan 4 terbaca genap, dan dengan begitu cabang ganjil ikut terbukti berjalan.

3.6 Lesson Learnt

Perbedaan i32 dan f64 bukan sekadar soal ada tidaknya koma desimal, melainkan soal operasi apa yang tersedia dan bagaimana hasilnya diperlakukan. Tipe i32 menyimpan bilangan bulat secara eksak tetapi pembagiannya memotong, terbukti dari $7 / 2$ yang menghasilkan 3, sedangkan f64 menyimpan nilai secara hampiran namun menyediakan sqrt, powf, dan abs. Sifat hampiran itu pula yang membuat $0.1 + 0.2$ tidak persis sama dengan 0.3, sehingga perbandingan dua f64 pada berkas ini selalu lewat hampir_sama() dengan ambang EPSILON. Konsekuensi praktisnya, seluruh komputasi vektor pada modul ini memakai f64, dan pencampuran tipe tidak akan lolos kompilasi tanpa cast eksplisit karena Rust tidak melakukan promosi tipe otomatis seperti C.

Immutability sebagai default membalik kebiasaan dari bahasa lain, sebab di Rust yang perlu ditulis justru izin untuk berubah, bukan larangan berubah. Praktiknya terasa saat tetap dan berubah diberi nilai awal sama tetapi hanya yang bertanda mut yang boleh menerima $+= 3$. Aturan ini tidak menambah biaya runtime sama sekali, tetapi hasilnya compiler bisa memastikan sebuah nilai tidak berubah diam-diam di tengah perhitungan. Const melangkah lebih jauh lagi, nilainya dipatok saat kompilasi dan memang tidak punya versi mutable.

Rust menolak $a + b$ pada dua array karena trait Add tidak diimplementasikan untuk $[T; N]$, dan itu keputusan desain, bukan kelalaian. Array hanyalah wadah elemen sejenis dengan panjang tetap, tanpa makna matematis yang melekat, jadi standard library tidak menebak apakah $+$ berarti penjumlahan element-wise, penggabungan, atau yang lain. Akibatnya operasi element-wise harus ditulis manual lewat iterator seperti $a.\text{iter}().\text{zip}(b).\text{map}(|(x, y)| x + y)$, dan itulah yang dikerjakan pada Task II. Keterbatasan ini sekaligus menjadi alasan nalgebra dipakai pada Task III, karena crate tersebut menyediakan tipe vektor yang sudah mengimplementasikan operator aritmetika lengkap dengan pemeriksaan dimensi di compile-time.

BAB IV VEKTOR DENGAN ARRAY DAN VEC

4.1 Deskripsi Program

Task II menyimpan dua vektor tiga dimensi sebagai array f64 bawaan Rust, lalu menghitung penjumlahan, perkalian skalar, dan hasil kali titik tanpa memakai crate apa pun. Vektor *u* berisi [1.0, 2.0, 3.0] dan *v* berisi [4.0, 5.0, 6.0], persis seperti yang diminta poin 1 modul. Seluruh operasi ditulis sendiri sebagai fungsi bebas di berkas `src/task2.rs`, bukan memanggil metode yang sudah jadi. Titik beratnya bukan pada hasil hitungannya, melainkan pada apa yang harus ditulis sendiri ketika bahasa tidak menyediakan tipe vektor matematis.

Semua fungsi hitung dibangun di atas iterator, bukan loop indeks. Pola yang dipakai berulang adalah `iter()` untuk membuka aliran elemen, `zip()` untuk memasangkan dua vektor, `map()` untuk menghitung per pasangan, lalu `collect()` atau `sum()` untuk menutupnya. Di luar operasi vektor, program juga menghitung empat statistik sederhana yaitu jumlah, rata, maks, dan mini sesuai poin 5 modul. Enam fungsi ini ditutup dengan modul uji ber-attribut `#[cfg(test)]` yang memverifikasi hasilnya terhadap nilai yang sudah diketahui.

4.2 Definisi Data yang Digunakan

Tipe `[f64; 3]` adalah array dengan panjang tetap yang panjangnya menjadi bagian dari tipe itu sendiri. Karena ukurannya diketahui saat kompilasi, seluruh isinya dialokasikan langsung di stack tanpa satu pun alokasi heap. Konsekuensinya, `[f64; 3]` dan `[f64; 4]` adalah dua tipe yang berbeda dan tidak bisa saling ditukar, sehingga fungsi yang menerima `[f64; 3]` hanya melayani vektor tiga dimensi saja. Karena f64 bertipe Copy, array ini juga Copy, jadi menyalinnya ke variabel lain tidak memindahkan kepemilikan.

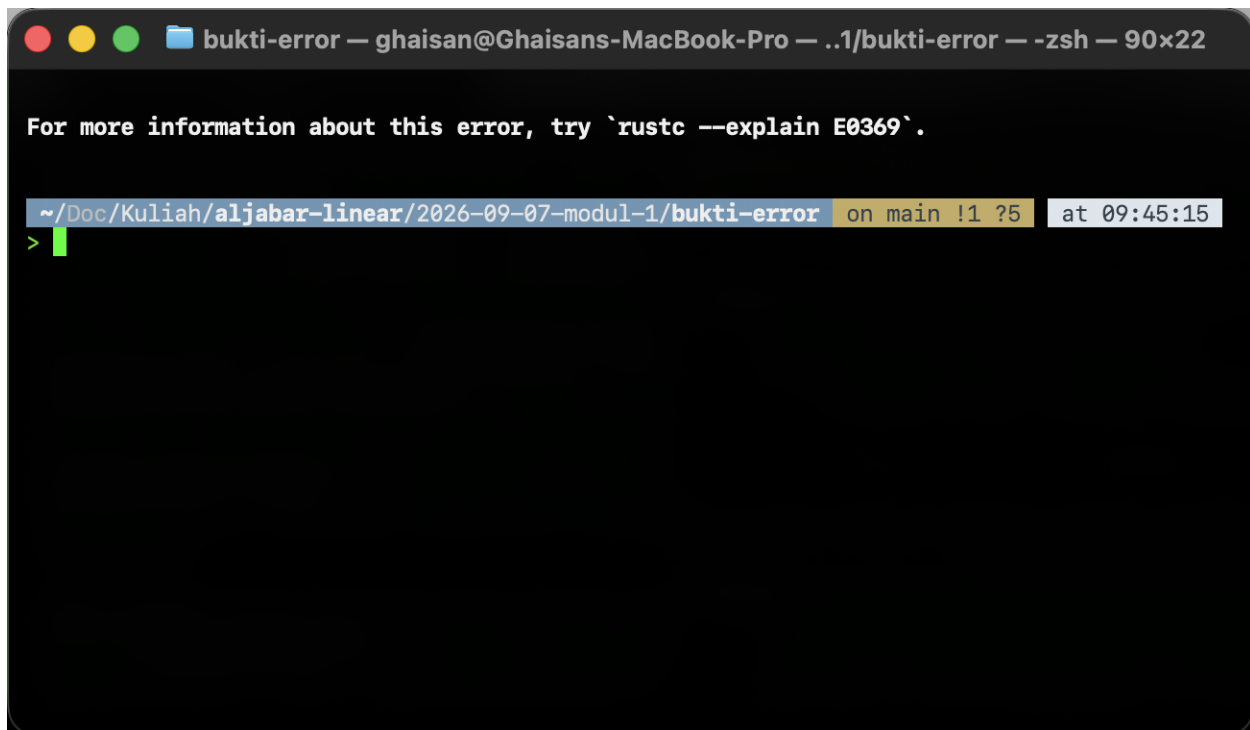
Agar fungsinya tidak terkunci pada satu panjang, semua parameter di `src/task2.rs` bertipe `&[f64]`, yaitu slice. Slice adalah borrow read-only yang tidak memiliki datanya, isinya cuma pointer ke elemen pertama ditambah panjangnya. Pemanggilan tambah(`&u`, `&v`) memberikan pinjaman atas array *u* dan *v*, bukan menyerahkan kepemilikannya, jadi kedua array masih bisa dipakai di baris berikutnya. Bentuk ini juga membuat satu fungsi yang sama menerima array panjang berapa pun maupun Vec, karena keduanya bisa dipinjam sebagai slice.

Tipe kembalian fungsi tambah dan skalar adalah `Vec<f64>`, buffer bertumbuh yang isinya berada di heap. Alasannya, `collect()` harus menghasilkan nilai yang dimiliki pemanggil, sedangkan slice tidak memiliki apa pun dan array butuh panjang yang sudah pasti saat kompilasi. Vec menampung berapa pun elemen yang keluar dari iterator dan panjangnya baru ditentukan saat runtime. Harganya adalah satu alokasi heap per pemanggilan, sesuatu yang tidak terjadi pada dot yang langsung meringkas semuanya menjadi satu f64.

4.3 Keterbatasan Array Native

Poin 2 modul meminta pembuktian bahwa $w = u + v$; ditolak kompilator, lengkap dengan pesan galat persisnya. Operator $+$ di Rust bukan sesuatu yang otomatis berlaku untuk sembarang tipe, melainkan sekadar gula sintaksis untuk metode `add` dari trait `std::ops::Add`. Pustaka standar tidak pernah mengimplementasikan `Add` untuk `[T; N]`, jadi tidak ada apa pun yang bisa dipanggil kompilator ketika dua array ditemui di kiri dan kanan tanda plus. Penolakannya terjadi murni di tahap pengecekan tipe, sebelum satu instruksi mesin pun dihasilkan.

Baris itu diaktifkan sebentar untuk merekam keluaran `rustc`, lalu dikembalikan menjadi komentar di `src/task2.rs` supaya proyeknya tetap bisa dibangun. Pesan yang keluar dari `rustc 1.98.1` adalah `error[E0369]: cannot add `[f64; 3]` to `[f64; 3]``, disusul potongan kode yang menandai kedua operand beserta tipenya masing-masing. Kompilator menunjuk langsung ke posisi tanda plus dan memberi label `[f64; 3]` di kiri dan di kanan, jadi tidak ada tebak-tebakan soal ekspresi mana yang bermasalah. Perbaikannya jelas sejak awal, yaitu menulis fungsi penjumlahan sendiri atau memakai tipe vektor dari pustaka numerik.



Gambar 4.1: Galat E0369 saat mencoba menjumlahkan dua array secara langsung

Perilaku ini kontras dengan Python yang tidak menolak apa pun pada kasus serupa. Ekspresi `[1, 2, 3] + [4, 5, 6]` di Python dijalankan lewat `list.__add__` dan menghasilkan `[1, 2, 3, 4, 5, 6]`, yaitu konkatenasi, bukan penjumlahan element-wise yang diinginkan. Tidak ada peringatan, tidak ada exception, dan panjang hasilnya berubah dari tiga menjadi enam. Kesalahannya baru terungkap

jauh di hilir ketika ada operasi lain yang memperlakukan panjang tersebut, dan pada perhitungan yang toleran terhadap panjang variabel kesalahan seperti ini bisa lolos sampai akhir tanpa pernah terdeteksi.

4.4 Kode Program

Berikut isi lengkap berkas src/task2.rs.

```
// Nama : Ghaisan Khoirul Badruzaman
// NIM : 251524048
// Kelas : 2B-D4
// Modul 1 - Task II: Vektor dengan Array & Vec (Manual)

/// Penjumlahan dua vektor.
///
/// Tabel 4 modul menyebut pemeriksaan dimensi untuk array native dilakukan manual
saat runtime,
/// jadi panjangnya dicek di sini. Tanpa assert, zip cuma berhenti di slice terpendek
dan
/// hasilnya salah tanpa peringatan apa pun.
pub fn tambah(u: &[f64], v: &[f64]) -> Vec<f64> {
    assert_eq!(u.len(), v.len(), "panjang kedua vektor harus sama");
    u.iter().zip(v).map(|(a, b)| a + b).collect()
}

pub fn skalar(k: f64, u: &[f64]) -> Vec<f64> {
    u.iter().map(|a| k * a).collect()
}

pub fn dot(u: &[f64], v: &[f64]) -> f64 {
    assert_eq!(u.len(), v.len(), "panjang kedua vektor harus sama");
    u.iter().zip(v).map(|(a, b)| a * b).sum()
}

pub fn jumlah(u: &[f64]) -> f64 {
    u.iter().sum()
}

/// Slice kosong menghasilkan NaN karena 0.0 dibagi 0.0.
pub fn rata(u: &[f64]) -> f64 {
    jumlah(u) / u.len() as f64
}

/// f64 tidak mengimplementasikan Ord (gara-gara NaN), jadi Iterator::max tidak bisa
dipakai.
/// Solusinya fold dengan f64::max. Slice kosong menghasilkan NEG_INFINITY.
pub fn maks(u: &[f64]) -> f64 {
    u.iter().copied().fold(f64::NEG_INFINITY, f64::max)
}

/// Slice kosong menghasilkan INFINITY, kebalikan dari maks.
pub fn mini(u: &[f64]) -> f64 {
    u.iter().copied().fold(f64::INFINITY, f64::min)
}

pub fn jalankan() {
    let u: [f64; 3] = [1.0, 2.0, 3.0];
    let v: [f64; 3] = [4.0, 5.0, 6.0];
```

```
// Poin 2 modul: array native tidak punya operator +, baris di bawah ditolak compiler.
```

```
// let w = u + v;
```

```
//
```

```
// Pesan persis dari rustc 1.98.1 waktu baris itu diaktifkan:
```

```
// error[E0369]: cannot add `[f64; 3]` to `[f64; 3]`
```

```
// |
```

```
// 4 | let w = u + v;
```

```
// | - ^ - [f64; 3]
```

```
// | |
```

```
// | [f64; 3]
```

```
//
```

```
// Beda dengan Python yang list + list malah jadi konkatenasi tanpa protes.
```

```
println!("u = {u:?}");
```

```
println!("v = {v:?}");
```

```
println!("u + v = {:?}", tambah(&u, &v));
```

```
println!("3u = {:?}", skalar(3.0, &u));
```

```
println!("u.v = {}", dot(&u, &v));
```

```
println!("sum = {}", jumlah(&u));
```

```
println!("mean = {}", rata(&u));
```

```
println!("max = {}", maks(&u));
```

```
println!("min = {}", mini(&u));
```

```
}
```

```
#[cfg(test)]
```

```
mod tests {
```

```
    use super::*;
```

```
    const EPS: f64 = 1e-12;
```

```
    fn dekat(a: &[f64], b: &[f64]) -> bool {
```

```
        a.len() == b.len() && a.iter().zip(b).all(|(x, y)| (x - y).abs() < EPS)
```

```
    }
```

```
    #[test]
```

```
    fn dot_ujian() {
```

```
        let u = [1.0, 2.0, 3.0];
```

```
        let v = [4.0, 5.0, 6.0];
```

```
        assert!((dot(&u, &v) - 32.0).abs() < EPS);
```

```
    }
```

```
    #[test]
```

```
    fn tambah_ujian() {
```

```
        let u = [1.0, 2.0, 3.0];
```

```
        let v = [4.0, 5.0, 6.0];
```

```
        assert!(dekat(&tambah(&u, &v), &[5.0, 7.0, 9.0]));
```

```
    }
```

```
    #[test]
```

```
    fn skalar_ujian() {
```

```
        let u = [1.0, 2.0, 3.0];
```

```
        assert!(dekat(&skalar(3.0, &u), &[3.0, 6.0, 9.0]));
```

```
        assert!(dekat(&skalar(0.0, &u), &[0.0, 0.0, 0.0]));
```

```
    }
```

```
    #[test]
```

```
    fn statistik_ujian() {
```

```

    let u = [1.0, 2.0, 3.0];
    assert!((jumlah(&u) - 6.0).abs() < EPS);
    assert!((rata(&u) - 2.0).abs() < EPS);
    assert!((maks(&u) - 3.0).abs() < EPS);
    assert!((mini(&u) - 1.0).abs() < EPS);
}

#[test]
#[should_panic(expected = "panjang kedua vektor harus sama")]
fn panjang_beda_ditolak() {
    dot(&[1.0, 2.0, 3.0], &[1.0, 1.0]);
}

#[test]
fn statistik_nilai_negatif() {
    let u = [-4.5, -1.0, -9.25];
    assert!((maks(&u) + 1.0).abs() < EPS);
    assert!((mini(&u) + 9.25).abs() < EPS);
}
}

```

4.5 Penjelasan Proses

Fungsi tambah dan skalar

Badan fungsi tambah hanya satu baris, yaitu `u.iter().zip(v).map(|(a, b)| a + b).collect()`. Panggilan `iter()` menghasilkan iterator yang mengeluarkan `&f64`, bukan `f64`, karena slice-nya cuma dipinjam. Metode `zip(v)` menggabungkannya dengan iterator dari `v` sehingga tiap langkah menghasilkan pasangan (`&f64`, `&f64`), lalu `map` menerapkan penjumlahan pada tiap pasangan. Rangkaian ini lazy dan tidak menghitung apa pun sampai `collect()` dipanggil, dan `collect` tahu harus membangun `Vec<f64>` karena tipe kembalian fungsinya sudah menyatakan itu. Fungsi skalar memakai pola yang sama tanpa `zip`, sebab k ditangkap oleh closure dari lingkungan sekitarnya.

Baris `assert_eq!(u.len(), v.len(), "panjang kedua vektor harus sama")` ditambahkan mengikuti Tabel 4 modul, yang menyatakan pemeriksaan dimensi pada array native dilakukan manual saat runtime. Tanpa penjaga itu, `zip` akan berhenti begitu iterator terpendek habis dan mengembalikan hasil sepanjang slice terpendek tanpa mengeluh sama sekali. Menjumlahkan vektor tiga elemen dengan vektor dua elemen akan menghasilkan `Vec` dua elemen yang secara matematis tidak berarti apa-apa. Kompilator tidak bisa menolongnya karena kedua parameter bertipe `&[f64]` yang panjangnya memang tidak dibawa oleh tipe, jadi satu-satunya tempat mendeteksi ketimpangan itu adalah saat runtime.

Fungsi dot

Alur dot identik dengan tambah sampai tahap map, bedanya penutupnya `sum()` dan bukan `collect()`. Setelah zip memasangkan elemen dan map mengalikannya, `sum()` meringkas seluruh aliran menjadi satu `f64` tunggal. Karena tidak ada `Vec` yang dibangun, fungsi ini sama sekali tidak menyentuh heap dan seluruh perhitungan berjalan di register. Penjaga `assert_eq!` yang sama dipasang di sini dengan alasan identik, sebab hasil kali titik atas dua vektor berbeda dimensi tidak terdefinisi dan zip akan diam-diam memotongnya. Untuk `u` dan `v` pada program ini hasilnya 1 kali 4 ditambah 2 kali 5 ditambah 3 kali 6, yaitu 32.

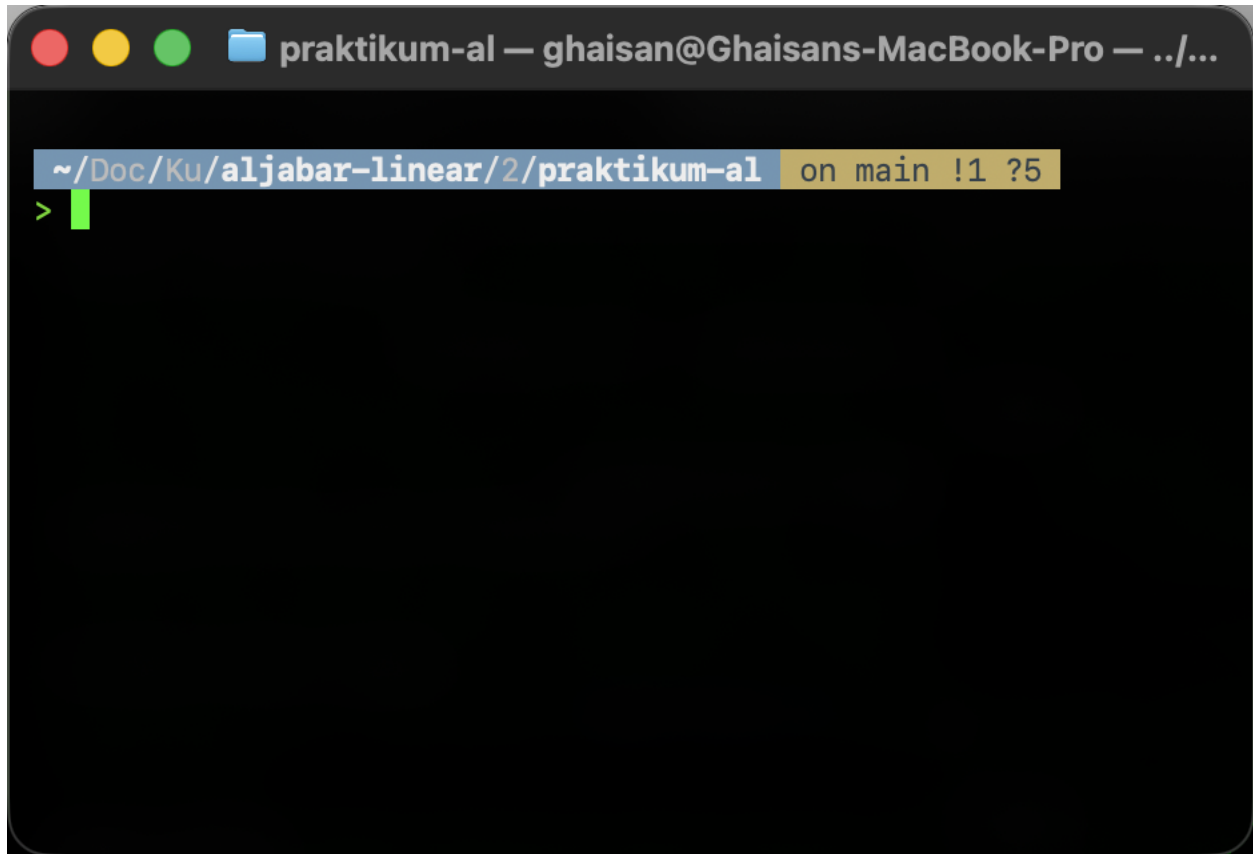
Fungsi statistik jumlah, rata, maks, dan mini

Fungsi jumlah cukup memanggil `u.iter().sum()` karena `f64` sudah punya implementasi `Sum`, sementara rata membagi hasilnya dengan `u.len()` as `f64`. Konversi as `f64` diperlukan sebab `len()` bertipe `usize` dan Rust tidak melakukan promosi tipe numerik secara otomatis. Efek sampingnya, slice kosong membuat rata menghitung 0.0 dibagi 0.0 dan mengembalikan `NaN`, bukan panic, sesuai aritmetika IEEE 754. Perilaku itu didokumentasikan sebagai komentar doc di atas fungsinya supaya pemanggil tahu apa yang terjadi pada kasus batas.

Fungsi maks dan mini tidak bisa memakai `Iterator::max` dan `Iterator::min` karena kedua metode itu mensyaratkan tipe elemen mengimplementasikan `Ord`. Tipe `f64` tidak mengimplementasikan `Ord`, hanya `PartialOrd`, sebab keberadaan `NaN` merusak sifat total order, yaitu `NaN` tidak lebih besar, tidak lebih kecil, dan tidak sama dengan nilai mana pun termasuk dirinya sendiri. Jalan keluarnya adalah fold, yang meringkas iterator memakai nilai awal dan sebuah fungsi penggabung. Pada maks nilai awalnya `f64::NEG_INFINITY` dengan penggabung `f64::max`, dan pada mini nilai awalnya `f64::INFINITY` dengan `f64::min`, jadi elemen pertama apa pun pasti mengalahkan seed-nya. Panggilan `copied()` disisipkan lebih dulu untuk mengubah aliran `&f64` menjadi `f64` agar cocok dengan tanda tangan `f64::max` yang menerima nilai, bukan referensi.

4.6 Output Program

Hasil eksekusi perintah `cargo run -- 2` ditampilkan pada gambar berikut.



Gambar 4.2: Hasil cargo run untuk Task II

Penjumlahan $u + v$ menghasilkan $[5.0, 7.0, 9.0]$ dan perkalian skalar $3u$ menghasilkan $[3.0, 6.0, 9.0]$, keduanya sama dengan nilai acuan yang dicantumkan modul pada poin 3. Hasil kali titik $u.v$ tercetak 32, sedangkan blok statistik memberi sum sebesar 6, mean sebesar 2, max sebesar 3, dan min sebesar 1 untuk vektor u . Kedua baris vektor tercetak dengan kurung siku dan angka berkoma desimal karena memakai format Debug lewat `{:?}`, sementara baris skalar memakai format Display sehingga 32.0 tampil sebagai 32 dan 2.0 tampil sebagai 2. Perbedaan tampilan ini murni soal formatter, nilainya tetap `f64` di dalam memori. Seluruh angka tersebut juga dijaga oleh modul uji, dan cargo test melaporkan 17 passed 0 failed untuk keseluruhan proyek.

4.7 Lesson Learnt

Menulis operasi vektor secara manual jelas lebih panjang daripada mengetik $u + v$. Tiga operasi dasar saja butuh tiga fungsi terpisah, ditambah dua baris `assert_eq!` yang tugasnya cuma mengulang apa yang seharusnya bisa dijamin oleh tipe. Namun kompensasinya terasa saat ada yang salah, karena kesalahan operator dilaporkan sebagai `error[E0369]` dengan tipe kedua operand ditunjuk langsung dan program tidak pernah sempat dijalankan. Kontrasnya dengan Python cukup tajam, di sana `list + list` berjalan mulus dan menghasilkan konkatenasi yang salah secara diam-diam. Verbositas di Rust dibayar sekali di awal, sementara kesalahan senyap di Python bisa dibayar berkali-kali saat debugging.

Sisi lemahnya juga terlihat jelas dari Task II ini. Kompilator memang menangkap penyalahgunaan operator, tetapi ia tidak menangkap ketidakcocokan dimensi sama sekali, sebab `&[f64]` sengaja membuang informasi panjang dari tipe. Akibatnya pemeriksaan dimensi turun ke runtime lewat `assert_eq!`, persis seperti yang dicatat Tabel 4 modul, dan program yang menjumlahkan dua vektor beda panjang baru gagal ketika baris itu benar-benar dieksekusi. Uji `panjang_beda_ditolak` dibuat memakai atribut `#[should_panic]` untuk memastikan penjaga itu memang bekerja. Jaminan compile-time yang sesungguhnya baru muncul kalau dimensinya kembali dibawa oleh tipe.

Untuk tiga sampai empat operasi pada vektor tiga elemen, pendekatan manual ini masih sepadan dan tidak menambah dependensi apa pun. Kebutuhan akan pustaka numerik mulai nyata begitu operasinya bertambah menjadi norma, normalisasi, hasil kali silang, proyeksi, sampai perkalian matriks, sebab setiap tambahan berarti satu fungsi baru beserta penjaga panjangnya sendiri. Pustaka seperti `nalgebra` memindahkan pemeriksaan dimensi kembali ke tipe lewat `SVector`, mengembalikan operator `+` dan `*` yang wajar, dan menyediakan tata letak memori yang lebih ramah SIMD. Task III kemudian memakai `crate` itu untuk membandingkan langsung kedua pendekatan pada persoalan yang sama.

BAB V INTRODUKSI NALGEBRA UNTUK VEKTOR

5.1 Deskripsi Program

Task III mengulang operasi vektor yang sama seperti Task II, tetapi seluruh perhitungannya diserahkan ke crate nalgebra versi 0.35.0. Program dijalankan lewat subperintah `cargo run -- 3` dan memanggil fungsi `jalankan()` pada berkas `src/task3.rs`. Isinya dibagi mengikuti urutan modul, mulai dari pembuatan vektor, aritmetika element-wise lewat operator, dot product bersama norma dan sudut, fungsi statistik, lalu dua latihan terapan di bagian akhir. Semua hasil dicetak ke terminal memakai formatter bawaan nalgebra yang menggambar vektor dalam bentuk kolom berkurung.

Selain fungsi utama, berkas ini memuat modul unit test yang berisi enam pengujian. Lima di antaranya memverifikasi hasil aritmetika, dot product, norma, statistik vektor a , statistik vektor w , serta norma dan sudut antara p dan q . Satu pengujian lagi mengerjakan poin E.3 modul, yaitu membuktikan identitas $u \cdot u$ sama dengan kuadrat norma u untuk vektor a . Perbandingan bilangan floating point di seluruh pengujian tidak memakai `assert_eq!` langsung melainkan selisih mutlak terhadap konstanta EPS bernilai $1e-10$, kecuali pada pengujian element-wise yang membandingkan `Vector3` secara utuh karena nilainya bulat dan bisa direpresentasikan persis.

5.2 Definisi Data yang Digunakan

Vektor a dan b dibuat dengan `Vector3::new(1.0, 2.0, 3.0)` dan `Vector3::new(4.0, 5.0, 6.0)`. `Vector3<f64>` di nalgebra bukan tipe tersendiri melainkan alias untuk `SVector<f64, 3>`, jadi angka 3 ikut menjadi bagian dari tipe. Konsekuensinya dimensi vektor terkunci di sistem tipe dan compiler yang memeriksanya, bukan kode program saat runtime. Pemanggilan `a.nrows()` tetap tersedia untuk membaca dimensi, tetapi nilainya sudah pasti 3 sejak kompilasi dan bukan hasil pengukuran panjang buffer seperti `len()` pada slice.

Bentuk generik `SVector<f64, N>` dipakai dua kali dengan N yang berbeda. Vektor b_4 dideklarasikan sebagai `SVector<f64, 4>` dan diisi dari array literal memakai `SVector::from([1.0, 2.0, 3.0, 4.0])`, sedangkan vektor w pada latihan terapan dideklarasikan sebagai `SVector<f64, 10>` dan diisi lewat `SVector::from_iterator((1..=10).map(f64::from))`. Keduanya berbagi konstruktor yang sama persis, hanya parameter N -nya yang berganti, dan compiler memonomorfisasi masing-masing menjadi kode terpisah dengan ukuran buffer tetap. Iterator $(1..=10)$ menghasilkan `i32`, jadi `f64::from` dipakai untuk mengonversinya sebelum masuk ke `from_iterator`.

Latihan terapan pertama memakai `Vector2<f64>`, alias untuk `SVector<f64, 2>`, dengan $p = \text{Vector2::new}(3.0, 4.0)$ dan $q = \text{Vector2::new}(4.0, 0.0)$. Pasangan angka ini dipilih modul karena p membentuk segitiga siku-siku 3-4-5 sehingga normanya bulat dan mudah diperiksa manual,

sementara q sejajar sumbu x sehingga sudut antara keduanya bisa dihitung ulang di kertas. Nilai kosinus perantaranya disimpan pada variabel `cos_pq` bertipe `f64`. Seluruh data di task ini bertipe `f64` tanpa kecuali, sebab operasi norma dan `acos` hanya masuk akal pada bilangan pecahan.

5.3 Kode Program

Isi lengkap berkas `src/task3.rs` adalah sebagai berikut.

```
// Nama : Ghaisan Khoirul Badruzaman
// NIM : 251524048
// Kelas : 2B-D4
// Modul 1 - Task III: Introduksi nalgebra untuk Vektor

use nalgebra::{SVector, Vector2, Vector3};

pub fn jalankan() {
    println!("=== Task III: Vektor dengan nalgebra ===");

    // A. Pembuatan vektor
    let a = Vector3::new(1.0, 2.0, 3.0);
    let b4: SVector<f64, 4> = SVector::from([1.0, 2.0, 3.0, 4.0]);
    println!("a = {a}");
    println!("dimensi a: {}", a.nrows());
    println!("b4 = {b4}");
    println!("dimensi b4: {}", b4.nrows());

    // B. Aritmetika element-wise lewat operator
    let b = Vector3::new(4.0, 5.0, 6.0);
    println!("a + b = {}", a + b);
    println!("a - b = {}", a - b);
    println!("2a = {}", 2.0 * a);
    println!("a o b = {}", a.component_mul(&b)); // perkalian Hadamard

    // C. Dot product, norma, dan sudut
    let dot_ab = a.dot(&b);
    let norm_a = a.norm();
    // anotasi : f64 wajib di sini, tanpa itu tipe hasil bagi masih generik
    // dan pemanggilan .acos() ditolak dengan E0282 type annotations needed
    let cos: f64 = a.dot(&b) / (a.norm() * b.norm());
    let sudut = cos.acos().to_degrees();
    println!("a . b = {dot_ab}");
    println!("||a|| = {norm_a}");
    println!("sudut a dan b = {sudut} derajat");

    // D. Fungsi statistik
    println!("sum = {}", a.sum());
    println!("mean = {}", a.mean());
    println!("max = {}", a.max());
    println!("min = {}", a.min());

    // E.1 Norma, dot product, dan sudut untuk dua vektor 2 dimensi
    let p = Vector2::new(3.0, 4.0);
    let q = Vector2::new(4.0, 0.0);
    let cos_pq: f64 = p.dot(&q) / (p.norm() * q.norm());
    println!("||p|| = {}, ||q|| = {}", p.norm(), q.norm());
    println!("p . q = {}", p.dot(&q));
    println!("sudut p dan q = {} derajat", cos_pq.acos().to_degrees());
```

```

// E.2 Vektor 10 dimensi berisi 1 sampai 10
let w: SVector<f64, 10> = SVector::from_iterator((1..=10).map(f64::from));
// dicetak dalam bentuk transpos supaya tidak memakan 10 baris terminal
println!("w = {}", w.transpose());
println!("jumlah w = {}", w.sum());
println!("rata-rata w = {}", w.mean());
println!("maksimum w = {}", w.max());
println!("minimum w = {}", w.min());
}

#[cfg(test)]
mod tests {
    use super::*;

    const EPS: f64 = 1e-10;

    // E.3 identitas u . u = ||u||^2, dibandingkan pakai toleransi karena floating
    point
    #[test]
    fn dot_diri_sendiri_sama_dengan_kuadrat_norma() {
        let a: Vector3<f64> = Vector3::new(1.0, 2.0, 3.0);
        assert!((a.dot(&a) - a.norm().powi(2)).abs() < EPS);
    }

    #[test]
    fn dot_product_dan_norma() {
        let a: Vector3<f64> = Vector3::new(1.0, 2.0, 3.0);
        let b: Vector3<f64> = Vector3::new(4.0, 5.0, 6.0);
        assert!((a.dot(&b) - 32.0).abs() < EPS);
        assert!((a.norm() - 14.0_f64.sqrt()).abs() < EPS);
    }

    #[test]
    fn operasi_element_wise() {
        let a: Vector3<f64> = Vector3::new(1.0, 2.0, 3.0);
        let b: Vector3<f64> = Vector3::new(4.0, 5.0, 6.0);
        assert_eq!(a + b, Vector3::new(5.0, 7.0, 9.0));
        assert_eq!(a - b, Vector3::new(-3.0, -3.0, -3.0));
        assert_eq!(2.0 * a, Vector3::new(2.0, 4.0, 6.0));
        assert_eq!(a.component_mul(&b), Vector3::new(4.0, 10.0, 18.0));
    }

    #[test]
    fn statistik_vektor_a() {
        let a: Vector3<f64> = Vector3::new(1.0, 2.0, 3.0);
        assert!((a.sum() - 6.0).abs() < EPS);
        assert!((a.mean() - 2.0).abs() < EPS);
        assert!((a.max() - 3.0).abs() < EPS);
        assert!((a.min() - 1.0).abs() < EPS);
    }

    #[test]
    fn statistik_vektor_w() {
        let w: SVector<f64, 10> = SVector::from_iterator((1..=10).map(f64::from));
        assert!((w.sum() - 55.0).abs() < EPS);
        assert!((w.mean() - 5.5).abs() < EPS);
        assert!((w.max() - 10.0).abs() < EPS);
        assert!((w.min() - 1.0).abs() < EPS);
    }
}

```

```

#[test]
fn norma_dan_sudut_p_q() {
    let p: Vector2<f64> = Vector2::new(3.0, 4.0);
    let q: Vector2<f64> = Vector2::new(4.0, 0.0);
    assert!((p.norm() - 5.0).abs() < EPS);
    assert!((q.norm() - 4.0).abs() < EPS);
    assert!((p.dot(&q) - 12.0).abs() < EPS);

    let cos: f64 = p.dot(&q) / (p.norm() * q.norm());
    assert!((cos.acos().to_degrees() - 53.1301).abs() < 1e-4);
}
}

```

5.4 Penjelasan Proses

Pembuatan Vektor

Bagian A memakai dua konstruktor dari Tabel 5 modul. `Vector3::new` menerima tiga argumen langsung dan cocok untuk dimensi kecil yang sudah diketahui, sementara `SVector::from` menerima array literal sehingga dimensinya ikut dari panjang array. Pada baris b4 anotasi `SVector<f64, 4>` ditulis eksplisit agar pembaca kode langsung tahu dimensinya tanpa menghitung elemen di dalam kurung siku. Cetakan `a.nrows()` dan `b4.nrows()` memperlihatkan 3 dan 4, dan keduanya adalah konstanta yang sudah diketahui compiler, berbeda dengan `len()` pada `Vec` yang baru terbaca saat program berjalan.

Aritmetika Element-wise lewat Operator

Bagian B menunjukkan perbedaan paling terasa dibanding Task II. nalgebra mengimplementasikan trait `Add`, `Sub`, dan `Mul` untuk tipe vektornya, jadi $a + b$ dan $a - b$ bisa ditulis apa adanya tanpa fungsi pembantu. Perkalian skalar ditulis $2.0 * a$, bukan $a * 2.0$, karena yang tersedia adalah implementasi `Mul<Vector3<f64>>` pada `f64` di sisi kiri. Operator overloading ini bukan sihir, hanya trait biasa yang diimplementasikan crate untuk tipenya sendiri, dan efeknya kode perhitungan jadi terbaca mirip notasi matematika di modul.

Baris terakhir bagian B memanggil `a.component_mul(&b)` yang menghasilkan `[4, 10, 18]`. Operasi ini adalah perkalian Hadamard, yaitu perkalian elemen ke elemen yang hasilnya tetap vektor berdimensi sama, bukan dot product yang hasilnya satu skalar. Keduanya sama-sama mengalikan pasangan elemen, bedanya dot product menjumlahkan hasil perkaliannya sehingga 4 ditambah 10 ditambah 18 menjadi 32. Nama `component_mul` dipakai nalgebra justru untuk mencegah kekeliruan itu, sebab kalau operator `*` dipakai antar dua vektor artinya jadi ambigu.

Dot Product, Norma, dan Sudut

Bagian C menghitung `a.dot(&b)` yang menerima referensi ke `b`, jadi `b` tidak dipindahkan kepemilikannya dan masih bisa dipakai di baris berikutnya. Nilai `a.norm()` adalah akar dari 14, dicetak penuh sebagai 3.7416573867739413. Sudut antara `a` dan `b` dihitung lewat rumus baku, kosinus sudut sama dengan dot product dibagi hasil kali kedua norma, lalu `acos()` dan `to_degrees()` mengubahnya ke derajat dan menghasilkan 12.933154491899135 derajat. Nilai ini masuk akal karena `a` dan `b` arahnya cukup berdekatan di ruang tiga dimensi.

Baris `let cos: f64` pada program ini tidak bisa ditulis tanpa anotasi. Metode `dot()` dan `norm()` milik `nalgebra` dideklarasikan generik atas tipe skalar yang mengimplementasikan `trait numerik crate` tersebut, sehingga hasil pembagian `a.dot(&b) / (a.norm() * b.norm())` masih berupa parameter tipe yang belum ditentukan compiler. Pada saat itu compiler belum tahu tipe konkretnya, dan karena `.acos()` bukan metode yang berasal dari `trait` generik tadi melainkan metode inheren milik `f64`, pemanggilannya ditolak dengan galat `E0282 type annotations needed`. Menuliskan `: f64` pada variabel `cos` memaksa inferensi tipe menutup celah itu, dan setelah tipenya pasti `f64` barulah `acos()` ikut terselesaikan.

Jebakan yang sama muncul lagi di modul unit test, meski gejalanya sedikit berbeda. Di dalam fungsi pengujian, vektor `a` dibuat hanya dari literal 1.0, 2.0, 3.0 tanpa ada operasi lain yang mengunci tipenya, sehingga compiler tidak punya petunjuk apakah itu `f32` atau `f64`. Perbaikannya cukup dengan menulis `let a: Vector3<f64> = Vector3::new(1.0, 2.0, 3.0)`, dan pola itu diterapkan konsisten pada seluruh deklarasi vektor di dalam tests, termasuk `b`, `p`, `q`, dan `w`. Setelah anotasi dipasang, cargo test lolos dengan 17 pengujian passed dan 0 failed untuk keseluruhan proyek.

Fungsi Statistik

Bagian D memanggil empat metode statistik bawaan `nalgebra` pada vektor `a`, yaitu `sum()`, `mean()`, `max()`, dan `min()`, yang berturut-turut menghasilkan 6, 2, 3, dan 1. Perbandingan dengan Task II terlihat jelas di sini. Pada implementasi manual, `max` dan `min` tidak bisa memakai `Iterator::max` karena `f64` tidak mengimplementasikan `Ord` akibat keberadaan `NaN`, sehingga harus dilipat sendiri dengan `fold` dan `f64::max`. `nalgebra` sudah menyembunyikan detail itu di balik satu pemanggilan metode, jadi empat baris statistik cukup ditulis apa adanya tanpa fungsi pembantu satu pun.

Latihan Terapan

Latihan pertama menghitung norma dan sudut untuk $p = (3, 4)$ dan $q = (4, 0)$. Programnya mencetak norma p sebesar 5 dan norma q sebesar 4, dot product p dan q sebesar 12, serta sudut antara keduanya 53.13010235415599 derajat. Angka 5 muncul dari akar 9 ditambah 16, dan sudut 53.13 derajat adalah sudut baku segitiga 3-4-5 sehingga hasilnya bisa diperiksa manual tanpa kalkulator. Pengujian `norma_dan_sudut_p_q` memverifikasi keempat nilai itu, dengan toleransi $1e-4$ khusus untuk sudutnya karena nilai acuan 53.1301 memang dibulatkan.

Latihan kedua membuat `w` berisi 1 sampai 10 lewat `SVector::from_iterator`, lalu mencetaknya dengan `w.transpose()` supaya keluarannya jadi satu baris mendatar dan tidak menghabiskan sepuluh baris terminal. Nilai statistiknya adalah jumlah 55, rata-rata 5.5, maksimum 10, dan minimum 1, seluruhnya sesuai deret aritmetika 1 sampai 10. Latihan ketiga tidak dicetak ke layar melainkan dikerjakan sebagai unit test `dot_diri_sendiri_sama_dengan_kuadrat_norma`, yang memeriksa selisih `a.dot(&a)` terhadap `a.norm().powi(2)` tetap di bawah EPS. Perbandingan pakai toleransi dipilih karena `norm()` melibatkan operasi akar kuadrat sehingga hasil kuadratnya belum tentu kembali persis ke 14 dalam representasi biner f64.

5.5 Output Program

Keluaran lengkap Task III saat dijalankan dengan `cargo run -- 3` terlihat pada gambar berikut.


```
praktikum-al — ghaisan@Ghaisans-Mac...

=== Task III: Vektor dengan nalgebra ===
a =

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$


dimensi a: 3
b4 =

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$


dimensi b4: 4
a + b =

$$\begin{bmatrix} 5 \\ 7 \\ 9 \end{bmatrix}$$


a - b =

$$\begin{bmatrix} -3 \\ -3 \\ -3 \end{bmatrix}$$

```

Gambar 5.1a: Hasil cargo run untuk Task III, bagian atas

```

2a      =

$$\begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$


a o b =

$$\begin{bmatrix} 4 \\ 10 \\ 18 \end{bmatrix}$$


a . b = 32
||a|| = 3.7416573867739413
sudut a dan b = 12.933154491899135 derajat
sum     = 6
mean    = 2
max     = 3
min     = 1
||p|| = 5, ||q|| = 4
p . q = 12
sudut p dan q = 53.13010235415599 derajat
w =

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \end{bmatrix}$$


jumlah w      = 55
rata-rata w   = 5.5
maksimum w    = 10
minimum w     = 1

```

```
~/Doc/Ku/aljabar-linear/2/praktikum-al on main !1 ?5
```

Gambar 5.1b: Hasil cargo run untuk Task III, bagian bawah

Vektor dicetak nalgebra dalam bentuk kolom berkurung sehingga satu vektor memakan beberapa baris terminal, kecuali w yang sengaja ditranspos. Bagian aritmetika memperlihatkan $a + b$ bernilai $[5, 7, 9]$, $a - b$ bernilai $[-3, -3, -3]$, $2a$ bernilai $[2, 4, 6]$, dan hasil Hadamard $a \circ b$ bernilai $[4, 10, 18]$, persis seperti kolom hasil pada Tabel 6 modul. Baris berikutnya menampilkan $a \cdot b$ sama dengan 32, norma a sebesar 3.7416573867739413 yang merupakan akar 14, serta sudut

antara a dan b sebesar 12.933154491899135 derajat. Blok statistik menutup dengan sum 6, mean 2, max 3, dan min 1, lalu bagian latihan terapan mencetak hasil p, q, dan w seperti yang sudah dibahas.

5.6 Perbandingan dengan Implementasi Manual

Tabel 4 modul menempatkan array native dan SVector berhadapan langsung, dan Task II serta Task III menunjukkan bedanya di kode nyata. Pada Task II, baris $u + v$ harus dikomentari karena rustc 1.98.1 menolaknya dengan error[E0369] cannot add [f64; 3] to [f64; 3], begitu pula perkalian skalar $k * a$ yang tidak punya implementasi Mul untuk array. Akibatnya semua operasi ditulis sebagai fungsi terpisah, mulai dari tambah, skalar, sampai dot, dan setiap fungsi yang menerima dua slice harus memasang `assert_eq!` sendiri untuk memeriksa panjangnya. Pemeriksaan itu baru berjalan saat runtime, jadi ketidakcocokan dimensi tetap lolos kompilasi dan baru ketahuan ketika program panik.

Task III membalik keadaan tersebut. SVector mendukung $a + b$ dan $2.0 * a$ lewat operator karena trait aritmetikanya sudah diimplementasikan crate, dan dimensi menjadi bagian dari tipe sehingga menjumlahkan Vector3 dengan SVector<f64, 4> ditolak compiler sebelum program sempat dijalankan. Tidak ada satu pun `assert_eq!` panjang di `src/task3.rs`, sebab pekerjaan itu sudah diambil alih sistem tipe. Tabel 4 juga mencatat performa keduanya setara karena zero-cost abstraction, artinya generik dan trait di nalgebra habis saat monomorfisasi dan tidak menyisakan biaya tambahan saat program berjalan, bahkan tata letak SVector lebih ramah SIMD karena ukurannya diketahui saat kompilasi.

5.7 Lesson Learnt

Keamanan tipe saat kompilasi di nalgebra bekerja karena dimensi ikut masuk ke dalam tipe, bukan disimpan sebagai angka di dalam struct. Vector3<f64> yang sebenarnya SVector<f64, 3> tidak akan pernah bisa dijumlahkan dengan SVector<f64, 4>, dan penolakannya terjadi di layar compiler, bukan di terminal pengguna. Bandingkan dengan Task II yang bergantung pada `assert_eq!` manual, di mana kesalahan panjang slice baru muncul sebagai panic setelah program terlanjur berjalan. Menggeser pemeriksaan dari runtime ke compile time menghilangkan seluruh kelas kesalahan itu tanpa menambah satu baris pun kode pemeriksaan.

Keringkasannya juga terasa. Task II membutuhkan tujuh fungsi bebas untuk penjumlahan, perkalian skalar, dot product, jumlah, rata-rata, maksimum, dan minimum, ditambah penanganan khusus karena f64 tidak mengimplementasikan Ord. Task III mengerjakan operasi yang sama tanpa satu pun fungsi pembantu, cukup memanggil metode yang sudah disediakan crate. Kode yang tidak ditulis adalah kode yang tidak perlu diuji dan tidak bisa salah, jadi pengujian di Task III fokus memverifikasi nilai hasilnya, bukan memverifikasi implementasi operasinya.

Sisi lain yang perlu diingat, kenyamanan itu datang bersama sistem tipe generik yang kadang menuntut anotasi eksplisit. Galat E0282 pada baris `cos` adalah contohnya, dan solusinya bukan mengubah logika melainkan memberi tahu compiler tipe apa yang dimaksud lewat : `f64`. Pola serupa berulang di modul test dan diselesaikan dengan let a: `Vector3<f64> = Vector3::new(...)`. Setelah pola ini dipahami, hambatannya hilang, dan pekerjaan tambahannya jauh lebih murah dibanding menulis serta menguji sendiri seluruh operasi vektor dari nol.

BAB VI PENGUJIAN OTOMATIS DENGAN CARGO TEST

6.1 Deskripsi Pengujian

Atribut `#[cfg(test)]` dipasang tepat di atas mod tests pada ketiga berkas task, dan efeknya modul itu hanya ikut dikompilasi ketika cargo test dijalankan. Pada cargo build maupun cargo run, kompilator melewati seluruh isi modul tersebut sehingga biner hasil build tidak membawa satu baris pun kode uji. Karena itu bebas menaruh pembantu khusus test di dalamnya, misalnya konstanta EPS dan fungsi dekat di `task2.rs`, tanpa membuat program utama membawa simbol yang tidak pernah dipanggil dan tanpa memicu peringatan `dead_code`. Baris `use super::*` di awal modul menarik seluruh item dari modul induk, jadi fungsi yang tidak ditandai pub sekalipun, seperti `klasifikasi` dan `hampir_sama` di `task1.rs`, tetap terlihat dari dalam test.

Modul meminta pengujian otomatis di dua tempat yang eksplisit. Poin 6 pada Task II menyuruh menambahkan unit test untuk dot dengan kasus yang hasilnya sudah diketahui, yaitu 32, memakai kerangka `#[cfg(test)]`. Poin E.3 pada Task III menyuruh memverifikasi identitas `u.u` sama dengan kuadrat norma `u` di dalam unit test, bukan sekadar dicetak ke layar. Alasan praktisnya, keluaran `println!` hanya dibaca sekali oleh mata dan tidak mengeluh apa pun kalau suatu saat rumusnya diubah salah, sedangkan `assert!` menyimpan ekspektasi itu dalam bentuk yang bisa dijalankan ulang kapan saja dan langsung gagal berisik ketika hasilnya melenceng. Rubrik penilaian pada Tabel 7 juga menempatkan unit test lengkap sebagai syarat nilai A untuk kriteria Vektor yang berbobot 30 persen.

6.2 Daftar Unit Test

Berkas `task1.rs` memuat lima test yang semuanya menyasar pemahaman sintaks dan sistem tipe. Test `klasifikasi_genap_ganjil` memeriksa fungsi klasifikasi pada 10.0 dan 40.0 yang harus mengembalikan "genap" serta 25.0 dan -3.0 yang harus mengembalikan "ganjil", termasuk nilai negatif karena sisa bagi `i64` pada bilangan negatif bisa ikut negatif. Test `hampir_sama_menoleransi_galat_float` memastikan pembanding float buatan sendiri menerima `0.1 + 0.2` sebagai 0.3 tetapi tetap menolak selisih sebesar 1.0 lawan 1.001, jadi ambangnya tidak longgar sampai kehilangan makna. Dua test berikutnya menguji aritmetika, `pembagian_i32_terpotong` menahan perilaku `7 / 2` yang bernilai 3 dan `7 % 2` yang bernilai 1 berdampingan dengan `7.0 / 2.0` yang bernilai 3.5, sementara `operasi_float_dasar` mengunci hasil `powf`, `sqrt`, dan sisa bagi pada `f64`. Terakhir, `indexing_dan_slicing` memeriksa panjang array, elemen pertama, elemen terakhir lewat `v.len() - 1`, dan isi slice `&v[1..4]` yang harus sama dengan `[20.0, 30.0, 40.0]`.

Berkas task2.rs memuat enam test untuk operasi vektor manual. Empat di antaranya menguji jalur normal, yaitu dot_ujian dengan hasil 32, tambah_ujian yang membandingkan hasil penjumlahan terhadap [5.0, 7.0, 9.0], skalar_ujian yang mengecek 3u sama dengan [3.0, 6.0, 9.0] sekaligus kasus pengali nol, dan statistik_ujian yang menahan jumlah 6, rata-rata 2, maksimum 3, serta minimum 1 untuk vektor [1.0, 2.0, 3.0]. Test statistik_nilai_negatif memakai data [-4.5, -1.0, -9.25] untuk memastikan maks dan mini benar-benar mengembalikan elemen vektor, bukan nilai awal fold berupa NEG_INFINITY dan INFINITY yang akan lolos begitu saja kalau datanya kebetulan positif semua. Satu test sisanya, panjang_beda_ditolak, memakai atribut should_panic untuk membuktikan bahwa assert_eq! pemeriksa panjang di dalam dot memang memicu panic ketika kedua slice tidak sama panjang, bukan diam-diam memangkas hasil mengikuti slice terpendek seperti perilaku zip tanpa penjaga. Argumen expected pada atribut itu mencocokkan pesan panic sehingga test tetap gagal seandainya panic-nya datang dari sebab lain.

```
#[test]
#[should_panic(expected = "panjang kedua vektor harus sama")]
fn panjang_beda_ditolak() {
    dot(&[1.0, 2.0, 3.0], &[1.0, 1.0]);
}
```

Berkas task3.rs juga memuat enam test, seluruhnya bekerja pada tipe vektor nalgebra. Test dot_product_dan_norma menahan hasil a.dot(&b) sebesar 32 dan a.norm() sebesar akar 14, operasi_element_wise membandingkan a + b, a - b, 2.0 * a, dan a.component_mul(&b) terhadap Vector3 yang diharapkan, lalu statistik_vektor_a dan statistik_vektor_w memeriksa sum, mean, max, dan min untuk vektor tiga dimensi [1, 2, 3] serta vektor sepuluh dimensi berisi 1 sampai 10 dengan hasil 55, 5.5, 10, dan 1. Test norma_dan_sudut_p_q menguji latihan terapan E.1, yakni norma p sebesar 5, norma q sebesar 4, dot product 12, dan sudut antar keduanya 53.1301 derajat dengan toleransi 1e-4 karena angka acuannya sendiri sudah dibulatkan empat desimal. Test dot_diri_sendiri_sama_dengan_kuadrat_norma adalah yang diminta langsung oleh poin E.3 modul, memverifikasi identitas u.u sama dengan kuadrat norma u dengan membandingkan a.dot(&a) terhadap a.norm().powi(2). Identitas itu benar secara matematis, tetapi jalur perhitungannya berbeda, sisi kanan melewati sqrt lalu dikuadratkan lagi, sehingga hasilnya tidak wajib identik bit per bit dan pembandingannya tetap memakai toleransi.

```
// E.3 identitas u . u = ||u||^2, dibandingkan pakai toleransi karena floating point
#[test]
fn dot_diri_sendiri_sama_dengan_kuadrat_norma() {
    let a: Vector3<f64> = Vector3::new(1.0, 2.0, 3.0);
    assert!((a.dot(&a) - a.norm().powi(2)).abs() < EPS);
}
```

6.3 Perbandingan Floating Point

Hampir seluruh `assert` pada ketiga berkas ditulis dalam bentuk `(hasil - harapan).abs() < ambang`, bukan `assert_eq!(hasil, harapan)`. Penyebabnya ada pada representasi `f64` yang menyimpan bilangan sebagai pecahan basis dua, sehingga desimal seperti 0.1 dan 0.2 tidak punya wakil persis dan hanya dibulatkan ke nilai terdekat yang bisa disimpan. Akibatnya `0.1 + 0.2` menghasilkan `0.30000000000000004`, bukan 0.3, dan `assert_eq!` terhadap 0.3 akan gagal padahal secara matematis jawabannya benar. Operasi seperti `sqrt`, `powf`, `acos`, dan `to_degrees` menambah pembulatan di tiap langkah, jadi galatnya menumpuk mengikuti panjang rantai perhitungan. Ambang yang dipakai adalah `1e-10` untuk konstanta `EPSILON` di `task1.rs` dan `EPS` di `task3.rs`, serta `1e-12` untuk `EPS` di `task2.rs`, keduanya jauh lebih besar dari galat pembulatan pada bilangan sekecil ini tetapi masih jauh lebih kecil dari selisih yang benar-benar menandakan kesalahan rumus.

Ada beberapa `assert_eq!` yang tetap dipertahankan karena nilainya memang eksak. Test `klasifikasi_genap_ganjil` dan `pembagian_i32_terpotong` membandingkan `&str` dan `i32`, dua tipe yang tidak mengenal galat pembulatan sama sekali. Test `operasi_element_wise` di `task3.rs` juga memakai `assert_eq!` langsung pada `Vector3` karena seluruh operannya bilangan bulat kecil yang punya wakil persis di `f64`, dan penjumlahan, pengurangan, serta perkalian di antaranya masih menghasilkan bilangan bulat yang persis pula, sehingga hasilnya deterministik tanpa sisa pembulatan. Pemilihan ambang juga tidak seragam, test sudut `p` dan `q` memakai `1e-4` yang jauh lebih longgar karena membandingkan hasil `acos` dalam derajat terhadap acuan 53.1301 yang sudah dipotong empat angka di belakang koma. Ambang yang terlalu ketat di posisi itu akan membuat test gagal bukan karena kodenya salah, melainkan karena angka acuannya kurang teliti.

6.4 Hasil Pengujian

Seluruh test dijalankan sekaligus dengan perintah `cargo test` dari akar proyek.

```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../praktikum-al — -zsh — 92x32

Finished `test` profile [unoptimized + debuginfo] target(s) in 0.01s
Running unittests src/main.rs (target/debug/deps/praktikum_al-259a931b5ecf0107)

running 17 tests
test task1::tests::hampir_sama_menoleransi_galat_float ... ok
test task1::tests::indexing_dan_slicing ... ok
test task1::tests::operasi_float_dasar ... ok
test task1::tests::klasifikasi_genap_ganjil ... ok
test task1::tests::pembagian_i32_terpotong ... ok
test task2::tests::dot_ujian ... ok
test task2::tests::skalar_ujian ... ok
test task2::tests::statistik_nilai_negatif ... ok
test task2::tests::statistik_ujian ... ok
test task2::tests::tambah_ujian ... ok
test task3::tests::dot_diri_sendiri_sama_dengan_kuadrat_norma ... ok
test task3::tests::dot_product_dan_norma ... ok
test task3::tests::norma_dan_sudut_p_q ... ok
test task3::tests::operasi_element_wise ... ok
test task3::tests::statistik_vektor_a ... ok
test task2::tests::panjang_beda_ditolak - should panic ... ok
test task3::tests::statistik_vektor_w ... ok

test result: ok. 17 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

~/Doc/Kuliah/aljabar-linear/2026-09-07-modul-1/praktikum-al on main !1 ?5 at 09:45:13
>
```

Gambar 6.1: Hasil cargo test, 17 test lolos

Keluarannya diawali baris running 17 tests dan ditutup baris test result: ok. 17 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s. Urutan test di layar tidak mengikuti urutan berkas maupun urutan penulisannya karena cargo menjalankan test secara paralel di beberapa thread, jadi yang lebih dulu selesai lebih dulu tercetak. Baris untuk panjang_beda_ditolak muncul dengan penanda tambahan should panic, tanda bahwa test itu dinyatakan lolos justru karena programnya panic seperti yang diharapkan. Waktu eksekusi yang terbaca 0.00 detik wajar mengingat dimensi vektor terbesar yang diuji hanya sepuluh, sehingga seluruh beban kerja habis sebelum pewaktunya sempat mencatat angka.

6.5 Kualitas Kode

Di luar kebenaran hasil, kode diperiksa dengan dua alat bawaan toolchain. Perintah `cargo build` selesai tanpa satu pun warning, artinya tidak ada import yang menganggur, fungsi yang tidak terpakai, maupun variabel yang dideklarasikan lalu dilupakan. Perintah `cargo clippy --all-targets` dengan `clippy 0.1.98` juga bersih, hanya mencetak baris `Checking` dan `Finished` tanpa lint apa pun. Bendera `--all-targets` dipakai supaya linter ikut memeriksa modul di dalam `#[cfg(test)]`, bukan hanya target biner, sebab tanpa bendera itu kode uji luput dari pemeriksaan. Satu-satunya penekanan lint di seluruh proyek adalah `#[allow(clippy::approx_constant)]` di atas binding `phi` pada `task1.rs`, yang memang dicontohkan modul karena `clippy` membaca 3.14159 sebagai pendekatan kasar dari `std::f64::consts::PI`.

Pemeriksaan format dilakukan dengan `cargo fmt --check` yang berhenti tanpa mencetak diff dan mengembalikan status keluar nol, tanda seluruh berkas sudah sesuai gaya baku `rustfmt` sehingga tidak ada indentasi atau pemenggalan baris yang perlu dirapikan lagi. Ketiga pemeriksaan ini menjawab dua kriteria rubrik pada Tabel 7 secara langsung. Kriteria Setup Environment berbobot 20 persen menetapkan nilai A untuk toolchain Rust dan `nalgebra` yang terkonfigurasi sempurna dengan `cargo build` tanpa warning, dan kondisi itu terpenuhi memakai `rustc 1.98.1`, `cargo 1.98.1`, `nalgebra 0.35.0`, serta edition 2024. Kriteria Dasar Sintaks dan Sistem Tipe yang juga berbobot 20 persen mensyaratkan kode lolos `cargo clippy` untuk nilai A, dan hasil `clippy` yang bersih pada seluruh target memenuhi syarat tersebut.

BAB VII KESIMPULAN

Keempat task Modul 1 sudah selesai dikerjakan dan seluruhnya berjalan di mesin sendiri, MacBook Pro M2 Pro dengan target aarch64-apple-darwin. Toolchain yang dipakai adalah rustc 1.98.1 dan cargo 1.98.1 pada edition 2024, ditambah crate nalgebra 0.35.0 yang keterpasangannya sudah dibuktikan lewat Task 0. Proyek lolos cargo build tanpa satu pun warning, cargo clippy --all-targets bersih, dan cargo fmt --check tidak menemukan selisih format. Pengujian otomatis menghasilkan 17 unit test lolos dan 0 gagal, terbagi lima test di task1.rs, enam di task2.rs, dan enam di task3.rs. Dengan itu keempat kriteria pada rubrik penilaian modul, mulai dari setup environment sampai perbandingan vektor manual dengan nalgebra, sudah terpenuhi secara terukur, bukan sekadar diklaim.

Sistem tipe Rust ternyata memindahkan cukup banyak pekerjaan pemeriksaan ke waktu kompilasi. Binding bersifat immutable secara default, sehingga variabel tetap pada Task I tidak bisa diubah nilainya sampai kata kunci mut ditambahkan pada berubah, dan compiler yang menolak duluan kalau aturan itu dilanggar. Pemeriksaan tipe yang statis juga membuat perbedaan i32 dan f64 tidak bisa disamarkan: $7 / 2$ pada i32 menghasilkan 3 karena bagian pecahannya dipotong, sementara .sqrt() sama sekali tidak tersedia untuk integer dan menuntut cast eksplisit ke f64. Kesalahan seperti operand bertipe salah, pembagian integer yang tidak disadari, atau pemanggilan metode yang tidak dimiliki suatu tipe berhenti di layar compiler, tidak sempat lolos jadi bug runtime yang baru ketahuan waktu program dipakai.

Representasi vektor memakai array native memberi gambaran jelas soal apa yang sebenarnya disediakan bahasa dan apa yang tidak. Baris let w = u + v; pada Task II ditolak dengan galat E0369 cannot add [f64; 3] to [f64; 3], jadi penjumlahan, perkalian skalar, dot product, dan seluruh fungsi statistik harus ditulis sendiri lewat iterator zip, map, dan collect. Konsekuensi lain yang lebih halus adalah pemeriksaan dimensi jatuh sepenuhnya ke tangan programmer, karena zip diam-diam berhenti di slice terpendek dan mengembalikan hasil yang salah tanpa peringatan apa pun, sehingga assert_eq! pada panjang kedua slice sengaja dipasang di tambah dan dot. Kasus tepi lain juga tidak diurus bahasa, misalnya f64 tidak mengimplementasikan Ord karena adanya NaN sehingga maks dan mini terpaksa memakai fold dengan f64::max dan f64::min, dan slice kosong menghasilkan NEG_INFINITY atau INFINITY, bukan panic.

Pindah ke nalgebra, kode yang sama menyusut drastis. Operator overloading membuat a + b, a - b, dan 2.0 * a langsung bisa ditulis apa adanya, sementara component_mul, dot, norm, sum, mean, max, dan min menggantikan fungsi manual yang di Task II ditulis satu per satu. Dimensi ikut terkunci di sistem tipe: Vector3 dan SVector<f64, 10> membawa panjangnya sebagai bagian dari tipe, jadi menjumlahkan dua vektor berbeda dimensi gagal saat kompilasi dan assert manual

seperti pada Task II tidak diperlukan lagi. Harganya adalah anotasi tipe eksplisit di beberapa tempat, karena hasil `dot()` dan `norm()` masih generik sehingga let `cos: f64 = a.dot(&b) / (a.norm() * b.norm())`; wajib diberi anotasi, tanpa itu pemanggilan `.acos()` ditolak dengan galat `E0282 type annotations needed`. Hasil numeriknya sendiri cocok dengan hitungan manual, dot product a dan b tetap 32, norma a bernilai 3.7416573867739413 yang merupakan akar 14, dan sudut antara $p = (3, 4)$ dan $q = (4, 0)$ keluar 53.13010235415599 derajat.

Pengujian otomatis memberi cara memverifikasi klaim di laporan tanpa harus membaca ulang keluaran terminal setiap kali kode disentuh. Ketujuh belas test dijalankan sekali lewat cargo test dan langsung menunjukkan modul mana yang rusak kalau ada perubahan yang meleset, termasuk test `panjang_beda_ditolak` yang memakai `should_panic` untuk memastikan assert dimensi benar-benar aktif, bukan sekadar tertulis. Perbandingan nilai floating point tidak pernah dilakukan dengan tanda sama dengan langsung, melainkan lewat selisih absolut terhadap toleransi, EPS bernilai $1e-12$ di Task II dan $1e-10$ di Task III, karena $0.1 + 0.2$ pun tidak persis sama dengan 0.3 pada representasi biner f64. Besar toleransi juga menyesuaikan ketelitian nilai acuan, misalnya uji sudut p dan q memakai $1e-4$ karena nilai pembandingnya hanya ditulis sampai 53.1301. Pengecualiannya adalah hasil yang memang eksak seperti $a + b$ atau $2.0 * a$, yang boleh dibandingkan dengan `assert_eq!` karena komponennya bilangan bulat kecil yang tersimpan tepat di f64.

Modul 1 berhenti pada vektor tunggal dan operasi dasarnya, tetapi pola kerjanya sudah terbentuk: tulis versi manualnya dulu supaya paham apa yang dihitung, ganti dengan nalgebra supaya ringkas dan aman dimensi, lalu kunci hasilnya dengan unit test bertoleransi. Modul berikutnya melanjutkan ke operasi vektor lanjutan dan matriks, tempat jumlah operasi dan kemungkinan salah dimensi meningkat jauh dibanding vektor tiga elemen. Di situ keuntungan pengecekan dimensi pada waktu kompilasi dan kebiasaan menulis test sejak awal mestinya terasa lebih besar lagi.